

计算机系统结构

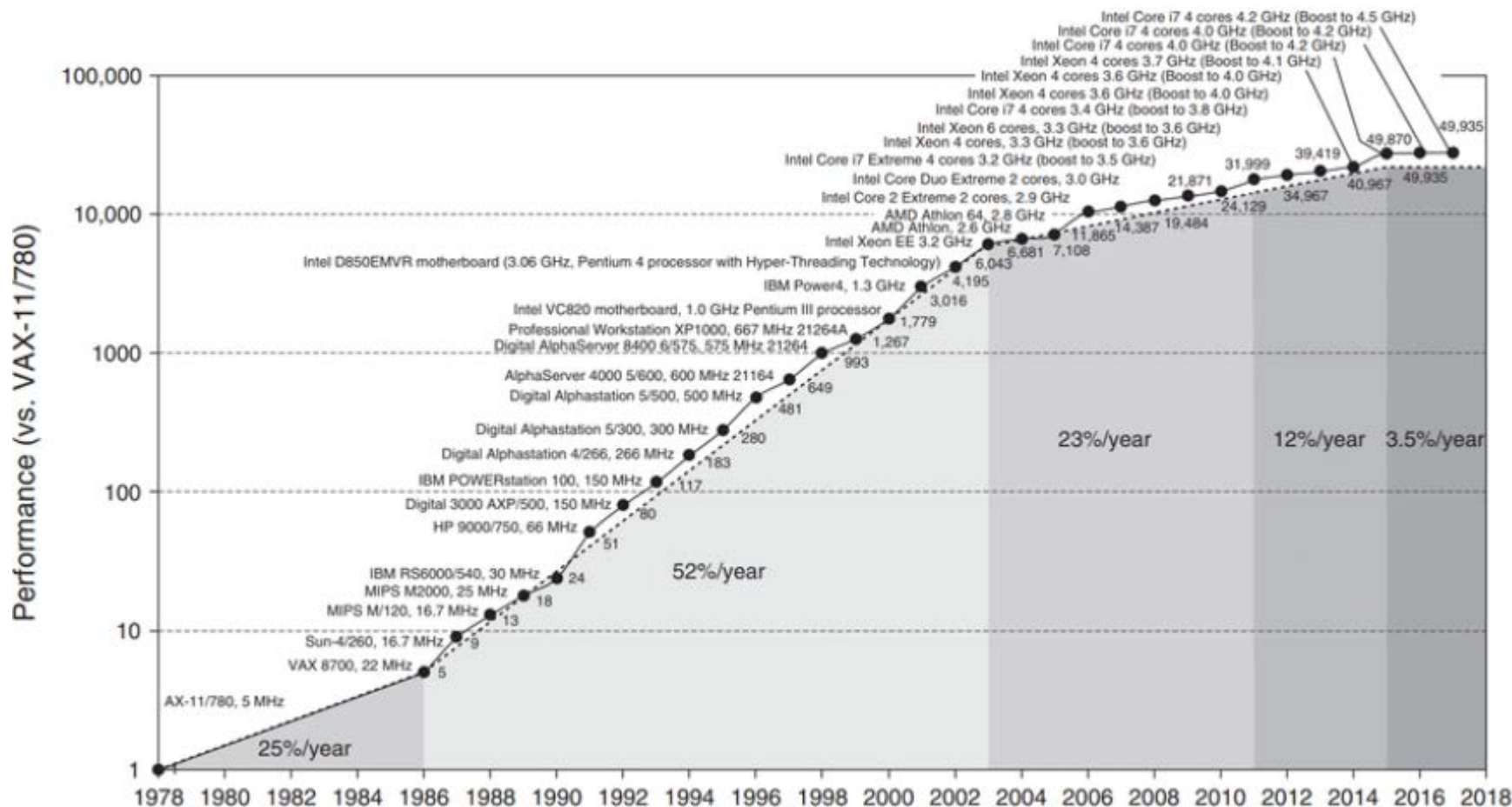
第10章 多处理机

目录

- 10.1 [引言](#)
- 10.2 [对称式共享存储器系统结构](#)
- 10.3 [分布式共享存储器系统结构](#)
- 10.4 [同步](#)
- 10.5 [同时多线程](#)
- 10.6 [大规模并行处理机](#)
- 10.7 [多核处理器及性能对比](#)
- 10.8 [多处理机实例](#)——Origin 2000

10.1 引言

1. 单处理器系统结构性能提升的边际效应不断降低

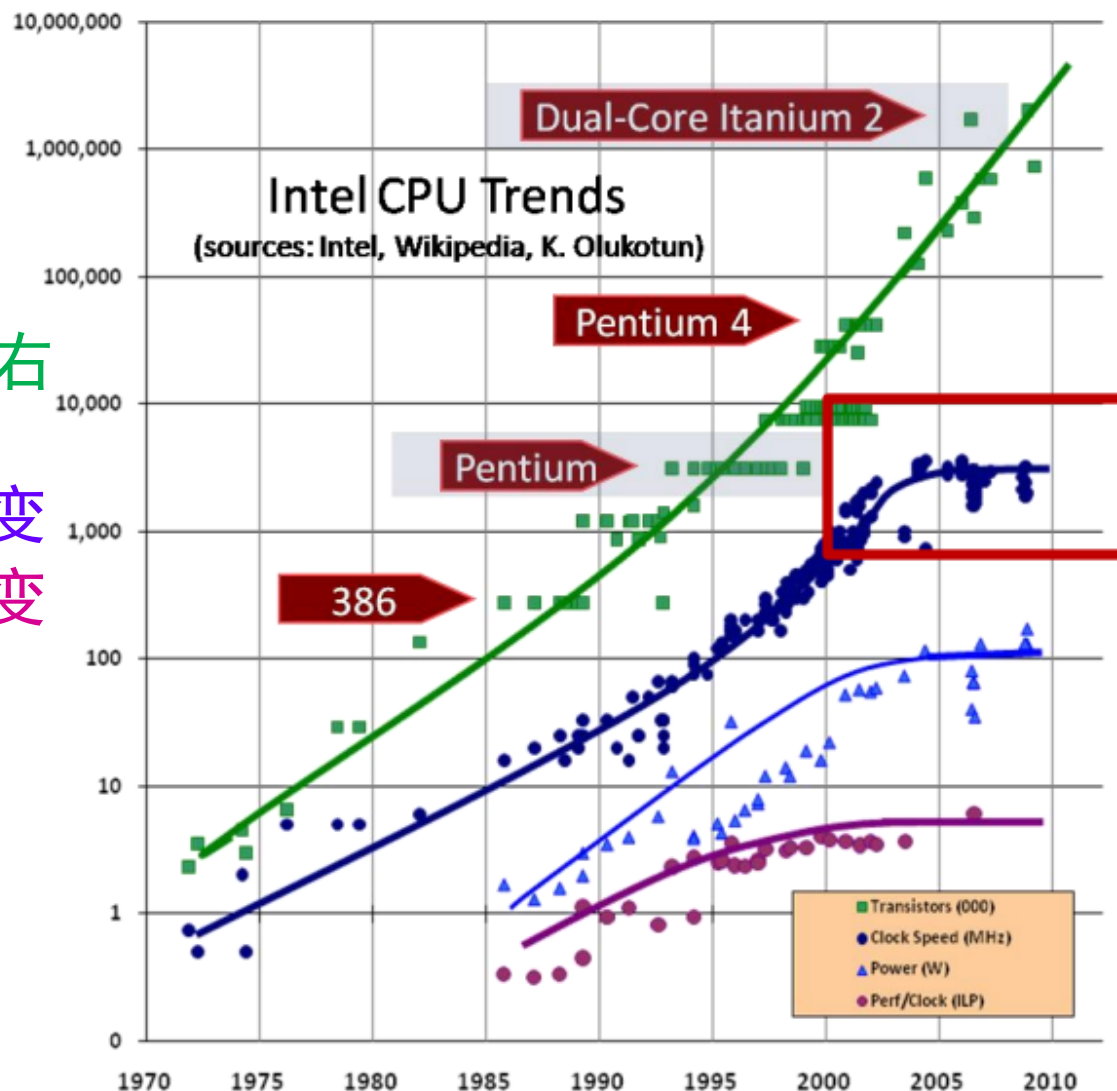


Growth in processor performance over 40 years

10.1 引言

2.通过增加处理器核数 提升整体计算能力

- (1) 摩尔定律推动晶体管数量每2年增加2倍左右
- (2) 时钟频率几乎不变
- (3) 处理器功率几乎不变
- (4) 指令并行度几乎不变



10.1 引言

- 多处理机 (Multiprocessor) 是指具有多个中央处理器 (CPU) 的计算机系统。
- 在多处理机系统中，多个 CPU 可以同时处理不同的任务或协作处理同一个任务，以提高系统的计算能力和处理效率。这些 CPU 共享计算机系统的资源，如内存、总线等，它们通过相互通信和协调来完成复杂的计算任务。
- 多处理机系统可以显著提升计算机的性能，适用于需要大规模数据处理、高性能计算等场景。
- 广义上讲，多核 (multi-core) 也是一种多处理机。

10.1 引言

3. 多处理机正起着越来越重要的作用。近几年来，人们确实开始转向了多处理机。

- IBM、SUN、AMD等公司开始多核多线程
- Intel于2004年宣布放弃了其高性能单处理器项目，转向多核（multi-core）的研究和开发。
- 并行计算机应用软件已有了稳定的发展。
- 充分利用商品化微处理器所具有的高性能价格比的优势。

4. 本章重点：中小规模的计算机（处理器的个数 <32 ）
（多处理机设计的主流）

10.1 引言

10.1.1 并行计算机系统结构的分类

★Flynn分类法

SISD、SIMD、MISD、MIMD

★MIMD已成为通用多处理机系统结构的选择，原因：

- MIMD具有灵活性；
- MIMD的商品化微处理器在性价比方面的优势。

计算机机群系统（cluster）是一类广泛被采用的MIMD机器。

从多核处理器、数据中心集群到大规模超算都使用这种思想。

3. 根据**存储器**的组织结构，把现有的**MIMD**机器分为两类：

1. 集中式共享存储器

2. 分布式存储器



每一类代表了一种存储器的结构和互连策略

➤ 集中式共享存储器结构

- 最多由几十个处理器构成。
- 各处理器共享一个集中式的物理存储器。

这类机器有时被称为**SMP机器（对称式共享存储器多处理机）**

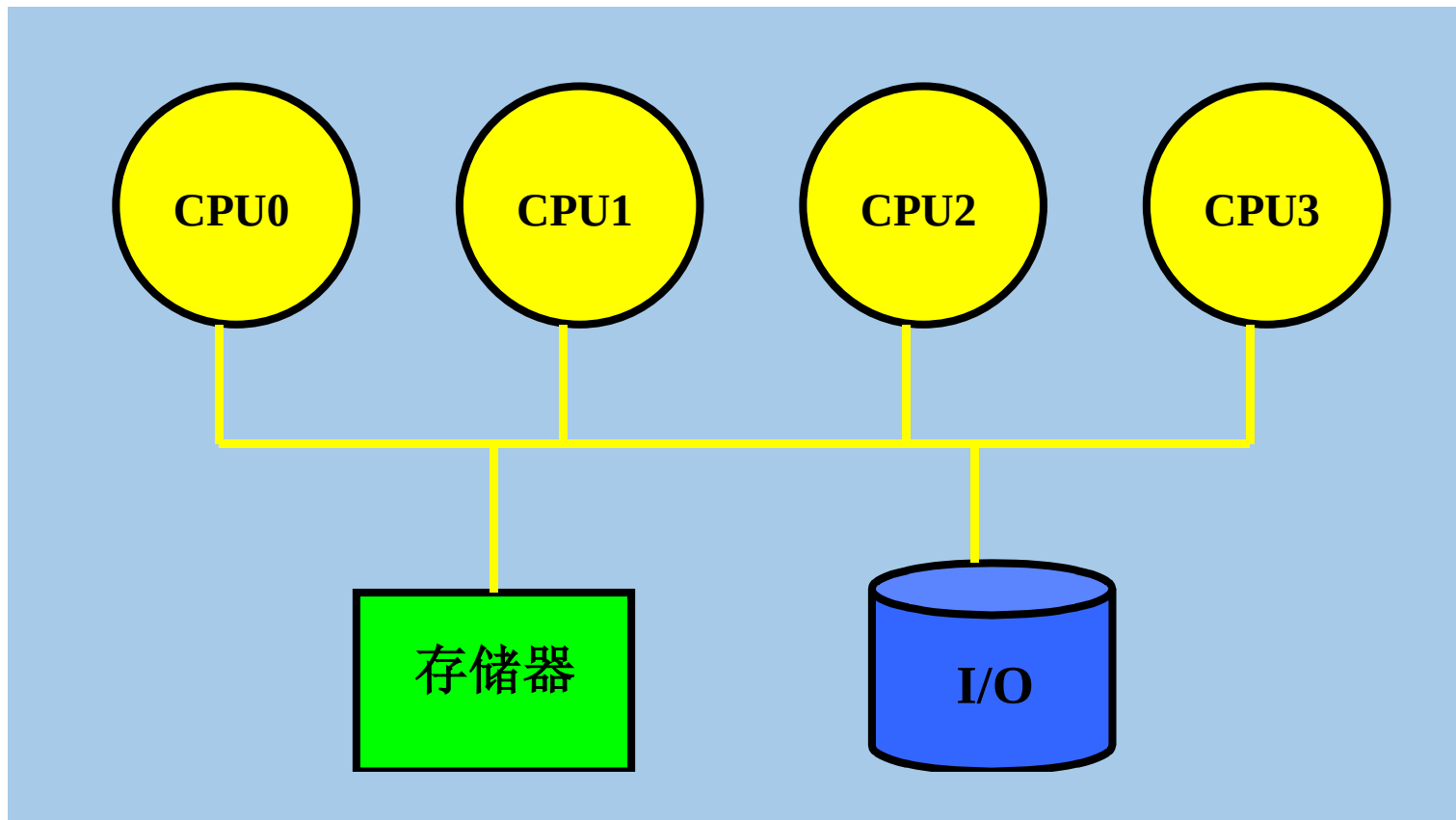
(Symmetric shared-memory MultiProcessor)

这种结构称为**UMA（Uniform Memory Access）结构**。

各个处理器访问存储器所花的时间相同。

SMP是目前最流行的结构。

10.1 引言



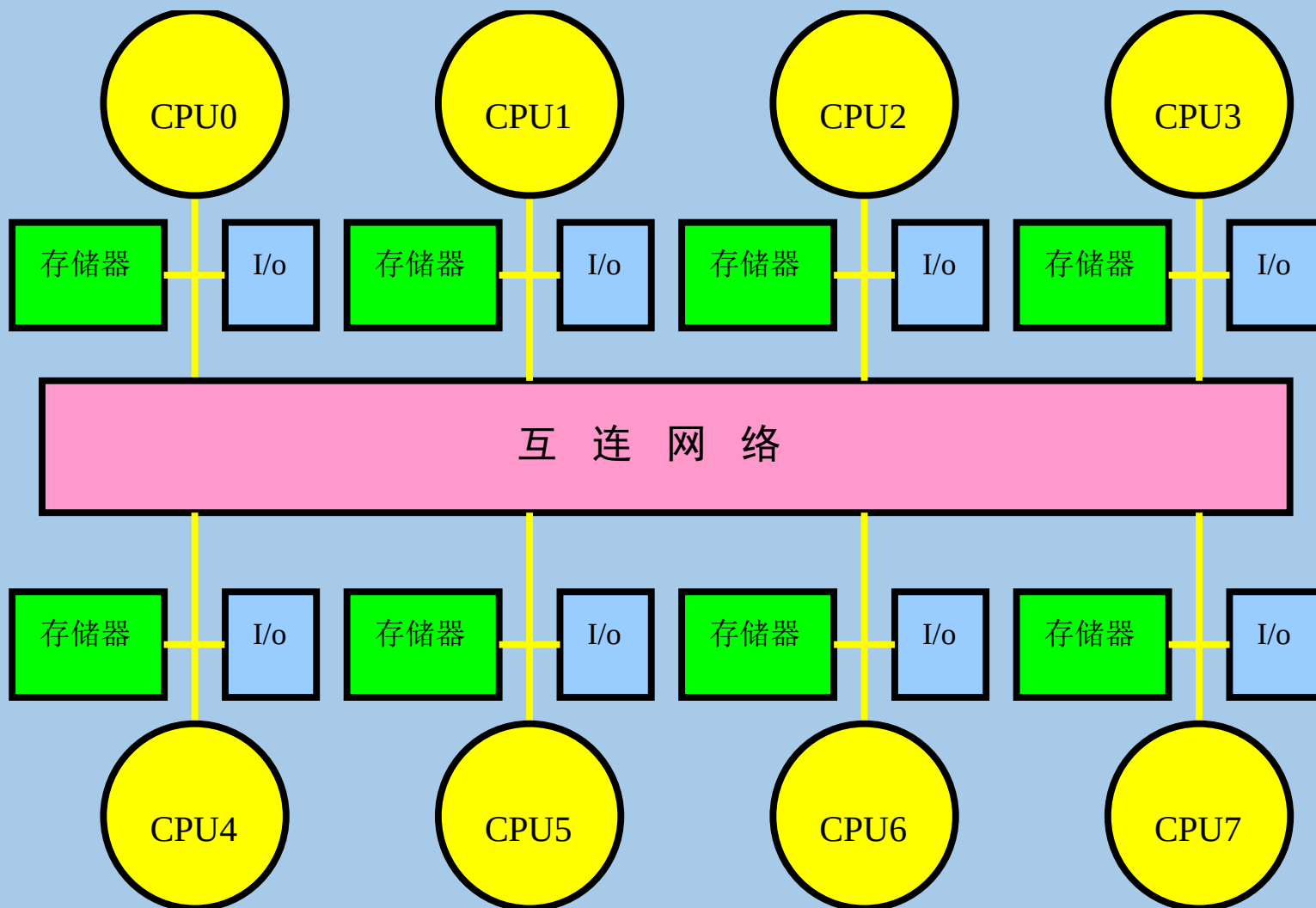
对称式共享存储器多处理机的基本结构

连接采用一条/多条总线或者交叉开关

10.1 引言

- 分布式存储器多处理机
 - 存储器在物理上是分布的。
 - 每个结点包含：
 - 处理器
 - 存储器
 - I / O
 - 互连网络接口

10.1 引言



10.1 引言

- 将存储器分布到各结点有两个优点
 - 如果大多数的访问是针对本结点的局部存储器，则可降低对存储器和互连网络的带宽要求；
 - 对本地存储器的访问延迟时间小。
- 最主要的缺点
 - 处理器之间的通信较为复杂，且各处理器之间访问延迟较大。
- 簇：超级结点
 - 每个结点内包含个数较少（例如2~8）的处理器；
 - 处理器之间可采用另一种互连技术（例如总线）相互连接形成簇。

10.1 引言

10.1.2 存储器系统结构和通信机制

1. 两种分布式存储器系统结构

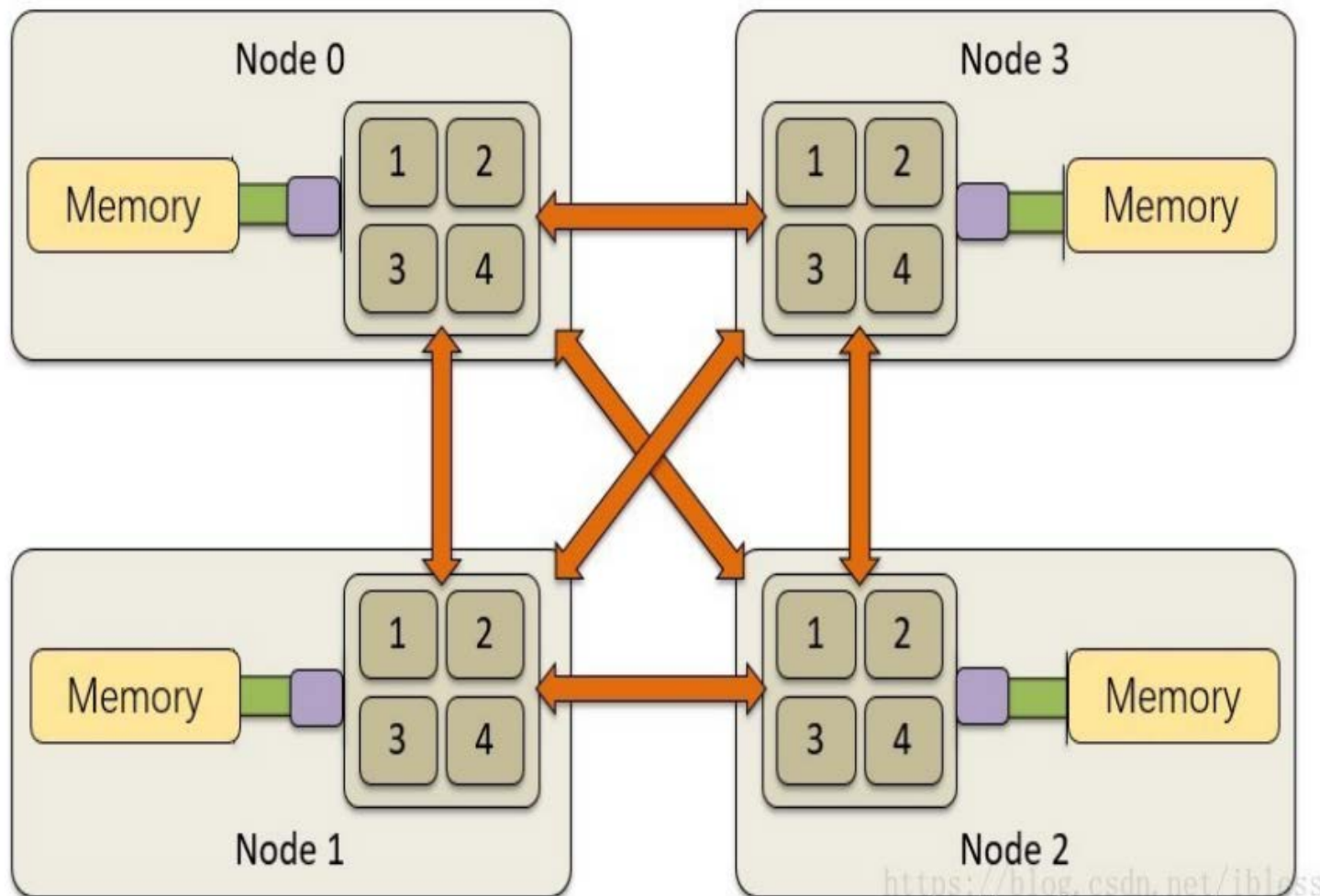
➤ 共享全局地址空间

- 物理上分离的所有存储器作为一个**统一的共享逻辑空间**进行编址。
- **分布式共享存储器系统 (DSM: Distributed Shared-Memory)** 也称为**NUMA**机器
(**NUMA: Non-Uniform Memory Access**, 非统一内存访问)
- 任何一个处理器可以访问该共享空间中的任何一个单元。

➤ 分布式的独立地址空间

- 整个系统的地址空间由多个独立的地址空间构成。
- 不同结点中的地址空间之间是相互独立的。
- 每个结点中的存储器只能由本地的处理器进行访问，远程的处理器不能直接对其进行访问。
- 每一个**处理器-存储器**模块实际上是一台单独的计算机。

10.1 引言



4路多核处理服务器结构

NUMA机器 (NUMA: Non-Uniform Memory Access)

2. 通信机制

➤ 共享存储器通信机制

- ❑ 共享地址空间的计算机系统采用
- ❑ 处理器之间是通过用load和store指令对相同存储器地址进行读/写操作来实现的。

➤ 消息传递通信机制

- ❑ 多个独立地址空间的计算机采用
- ❑ 通过处理器间显式地传递消息来完成
- ❑ 消息传递多处理机中，处理器之间是通过发送消息来进行通信的，这些消息请求进行某些操作或者传送数据。

10.1 引言

例如：一个处理器要对远程存储器上的数据进行访问或操作：

- 发送消息，请求传递数据或对数据进行操作；

可以把该消息看成一个**远程进程调用**

(RPC, Remote Process Call)

- 目的处理器接收到消息以后，执行相应的操作或代替远程处理器进行访问，并发送一个应答消息将结果返回。

□ 同步消息传递

请求处理器发送一个消息后一直要等到应答结果才继续运行。

□ 异步消息传递

数据发送方知道别的处理器需要数据，通信也可以从数据发送方开始，数据可以不经请求就直接送往数据接受方。

3. 不同通信机制的优点

➤ 共享存储器通信的主要优点

- ❑ 与常用的对称式（集中式）多处理机使用的通信机制兼容。
- ❑ 易于编程，同时在简化编译器设计方面也占有优势。
- ❑ 采用大家所熟悉的共享存储器模型开发应用程序，把重点放到解决对性能影响较大的数据访问上。
- ❑ 当通信数据量较小时，通信开销较低，带宽利用较好。
- ❑ 可以通过采用Cache技术来减少远程通信的频度，减少了通信延迟以及对共享数据的访问冲突。

10.1 引言

- 消息传递通信机制的**主要优点**
 - ❑ 硬件较简单。
 - ❑ 通信是显式的，因此更容易搞清楚何时发生通信以及通信开销是多少。
 - ❑ 显式通信可以让编程者重点注意并行计算的主要通信开销，使之有可能开发出结构更好、性能更高的并程序。
 - ❑ 同步很自然地与发送消息相关联，能减少不当的同步带来错误的可能性。

10.1 引言

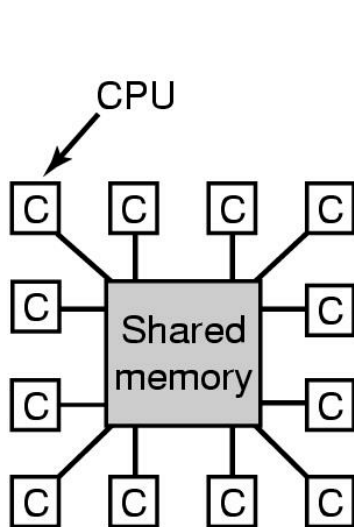
- 可在支持上面任何一种通信机制的硬件模型上建立所需的通信模式平台。
 - 在共享存储器上支持消息传递相对简单。
 - 在消息传递的硬件上支持共享存储器就困难得多。
所有对共享存储器的访问均要求操作系统提供地址转换和存储保护功能，即将存储器访问转换为消息的发送和接收。

现在通过总线连接的对称式共享存储器计算机占主导地位，但从长远看，在技术上的趋势是朝着中等规模的分布式共享存储器计算机方向发展。

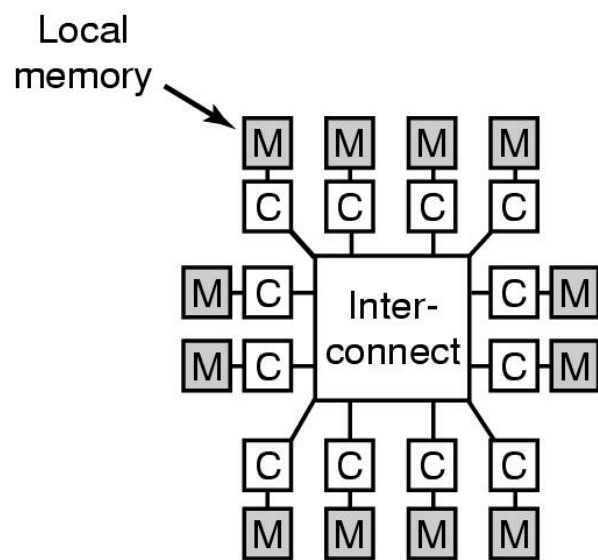
UMA & NUMA

1. Single address space visible to all CPUs
2. Access to remote memory via commands
 - LOAD
 - STORE
3. NUMA access to remote memory slower than to **local**

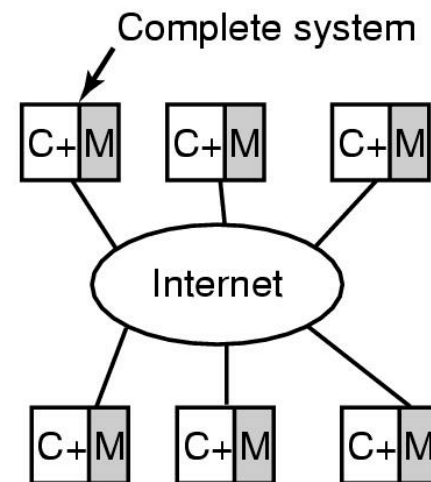
Multiprocessor Systems



(a)



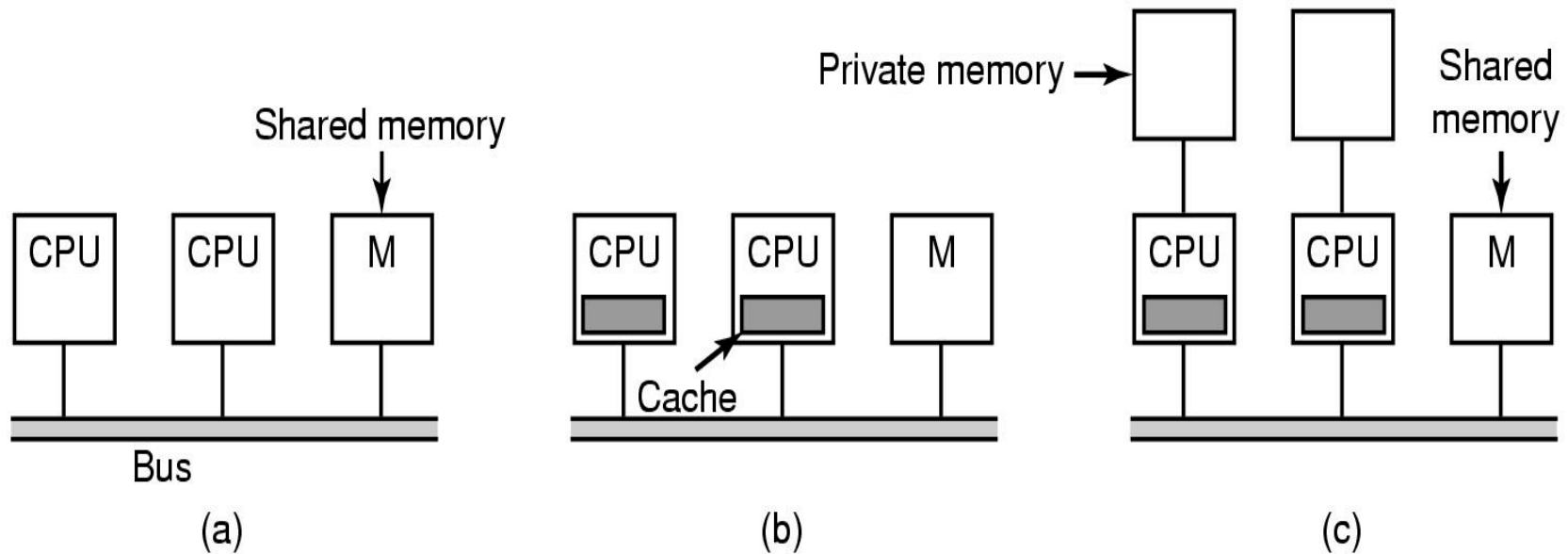
(b)



(c)

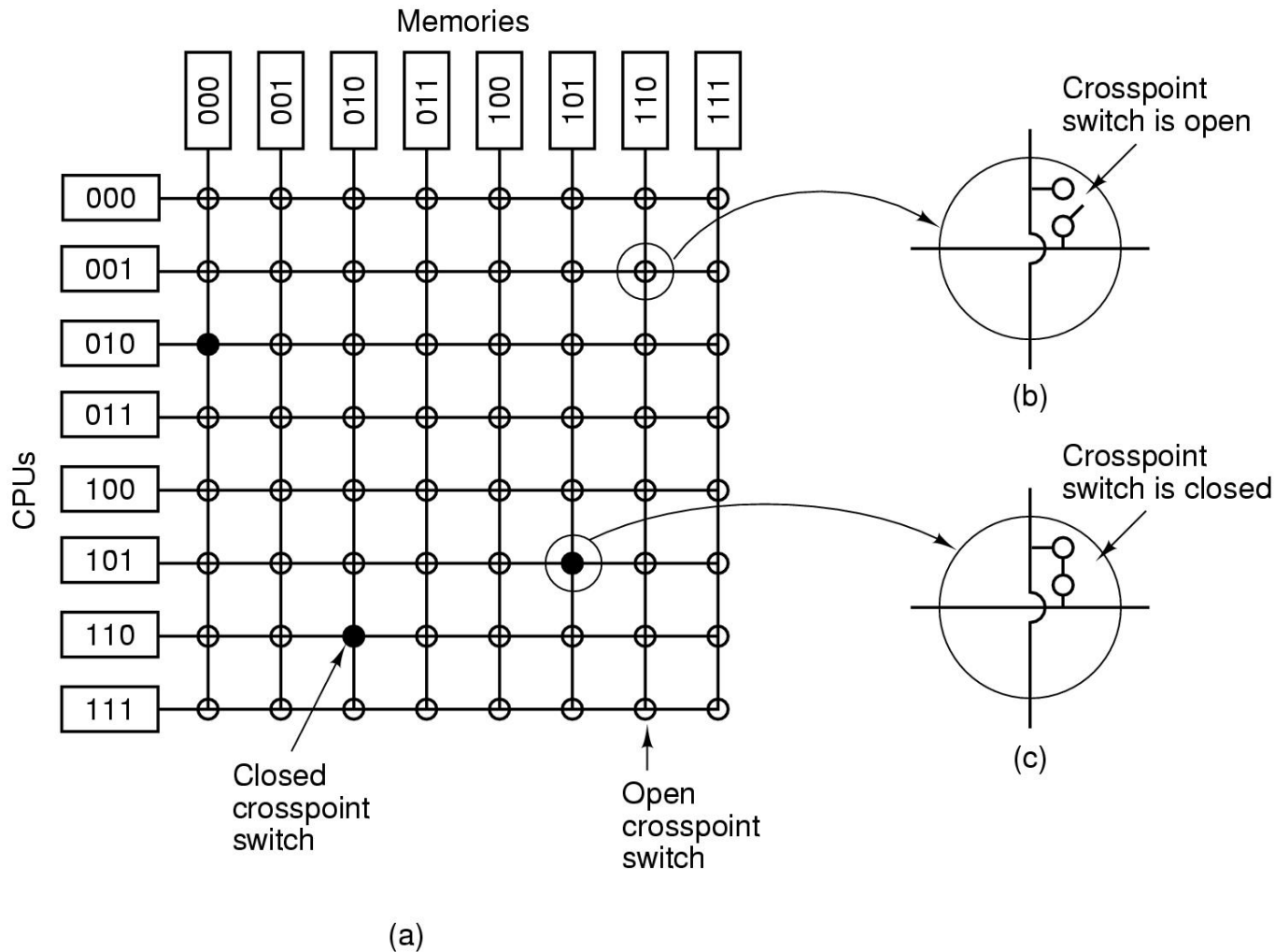
- (a) shared memory model
- (b) message passing multiprocessor
- (c) wide area distributed system

Multiprocessor Systems



Bus-based multiprocessors

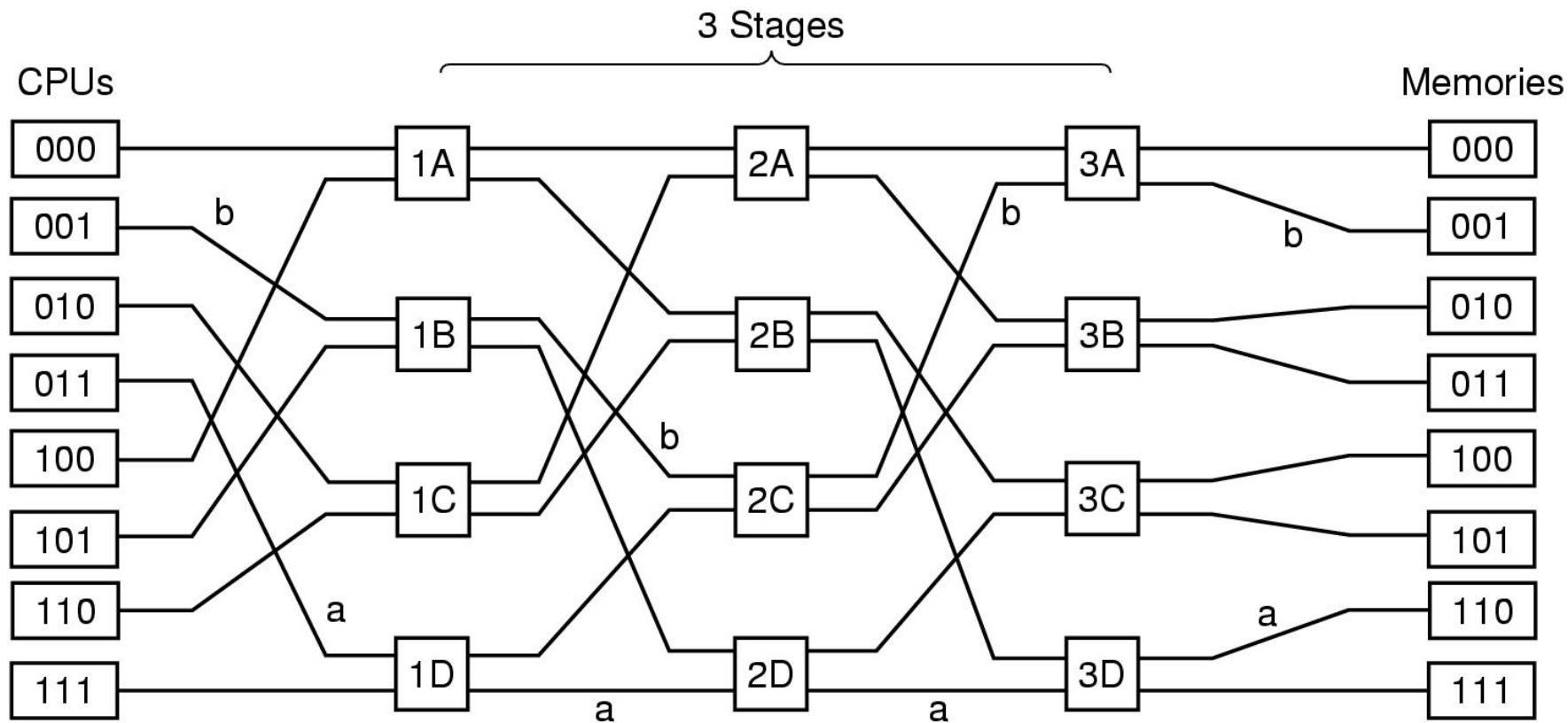
Multiprocessor Systems



UMA Multiprocessor using a crossbar switch

10.1引言

Multiprocessor Systems



Omega Switching Network

10.1 引言

全局共享地址空间

UMA

物理共享
逻辑共享

NUMA

物理分离
逻辑共享

集中式
共享存储
多处理机
SMP

分布式
共享存储
多处理机
DSM

共享存储器通信机制

分布式独立地址空间

物理共享
逻辑分区

物理分离
逻辑分区

集中式
共享存储
多处理机

分布式
存储器
多处理机

消息传递通信机制

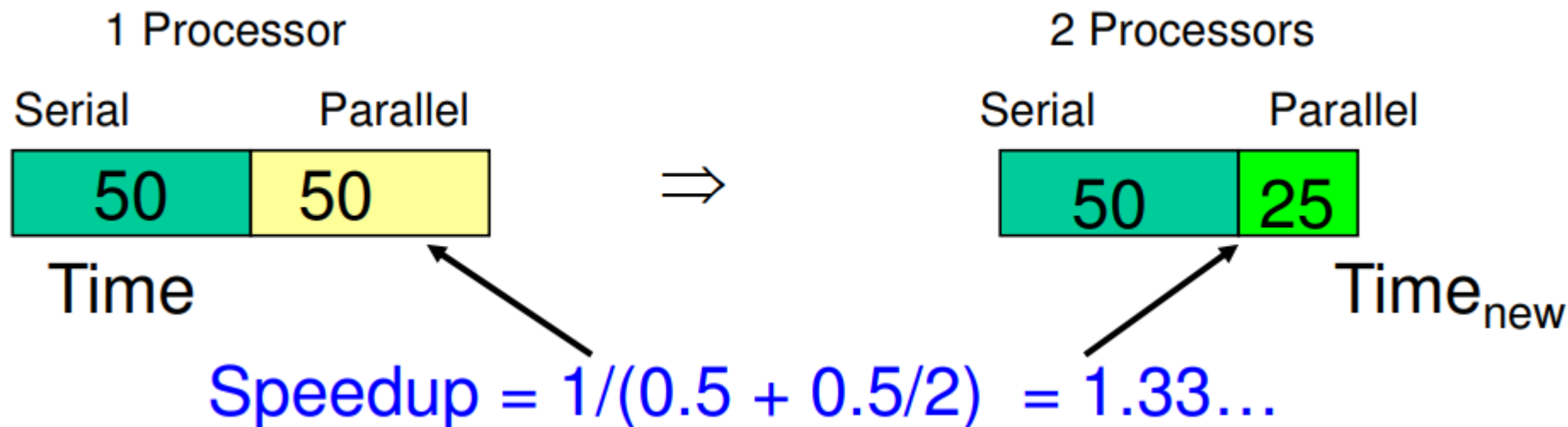
10.1 引言

10.1.3 并行处理面临的挑战

- 程序中的并行性有限
- 相对较大的通信开销

Amdahl's law

系统加速比 =
$$\frac{1}{(1 - \text{可加速部分比例}) + \frac{\text{可加速部分比例}}{\text{理论加速比}}}$$



10.1 引言

1. 第一个挑战

有限的并行性使计算机要达到很高的加速比十分困难。

例10.1 假设想用100个处理器达到80的加速比，求原计算程序中串行部分最多可占多大的比例？

解：Amdahl定律为：

$$\text{加速比} = \frac{1}{\frac{\text{可加速部分比例}}{\text{理论加速比}} + (1 - \text{可加速部分比例})}$$

$$80 = \frac{1}{\frac{\text{并行比例}}{100} + (1 - \text{并行比例})}$$

由上式可得：并行比例 = 0.9975 = 99.75%

即串行计算部分只能占0.25%。

2. 第二个挑战：多处理机中远程访问的延迟较大

- 在现有的机器中，处理器之间的数据通信大约需要50~1000个时钟周期。
- 主要取决于：
通信机制、互连网络的种类和机器的规模
- 在几种不同的共享存储器并行计算机中远程访问一个字的典型延迟如下表。

10.1 引言

远程访问一个字的延迟时间

机器	通信机制	互连网络	处理机最大数量	典型远程存储器访问时间 (ns)
Sun Starfire servers	UMA	多总线	64	500
SGI Origin 3000	NUMA	胖超立方体	512	500
Cray T3E	NUMA	3维环网	2048	300
HP V series	UMA	8×8交叉开关	32	1000
HP AlphaServer GS	UMA	开关总线	32	400

10.1 引言

例10.2 假设有一台32个处理器的多处理机，对远程存储器访问时间为200ns。除了通信以外，假设所有其它访问均命中局部存储器。当发出一个远程请求时，本处理器挂起。处理器的时钟频率为2GHz，如果指令基本的CPI为0.5（设所有访存均命中Cache），求在没有远程访问的情况下和有0.2%的指令需要远程访问的情况下，前者比后者快多少？

解：有0.2%远程访问的机器的实际CPI为：

$$\begin{aligned}\text{CPI} &= \text{基本CPI} + \text{远程访问率} \times \text{远程访问开销} \\ &= 0.5 + 0.2\% \times \text{远程访问开销}\end{aligned}$$

远程访问开销为：

$$\text{远程访问时间/时钟周期时间} = 200\text{ns} / 0.5\text{ns} = 400\text{个时钟周期}$$

$$\therefore \text{CPI} = 0.5 + 0.2\% \times 400 = 1.3$$

因此在没有远程访问的情况下的机器速度是有0.2%远程访问的机器速度的 $1.3/0.5=2.6$ 倍。

并行处理问题的解决：

- 并行性不足：采用并行性更好的算法
- 远程访问延迟的降低：靠系统结构支持和编程技术

3. 在并行处理中，影响性能（负载平衡、同步和存储器访问延迟等）的关键因素常依赖于：

应用程序的高层特性，如数据的分配，并行算法的结构以及在空间和时间上对数据的访问模式等。

- 依据应用特点可把多机工作负载大致分成两类：
 - 单个程序在多处理机上的并行工作负载
 - 多个程序在多处理机上的并行工作负载

4. 并行政程序的计算 / 通信比率

- 反映并行政程序性能的一个重要的度量：

计算与通信的比率

- 计算 / 通信比率随着处理数据规模的增大而增加；
随着处理器数目的增加而减少。

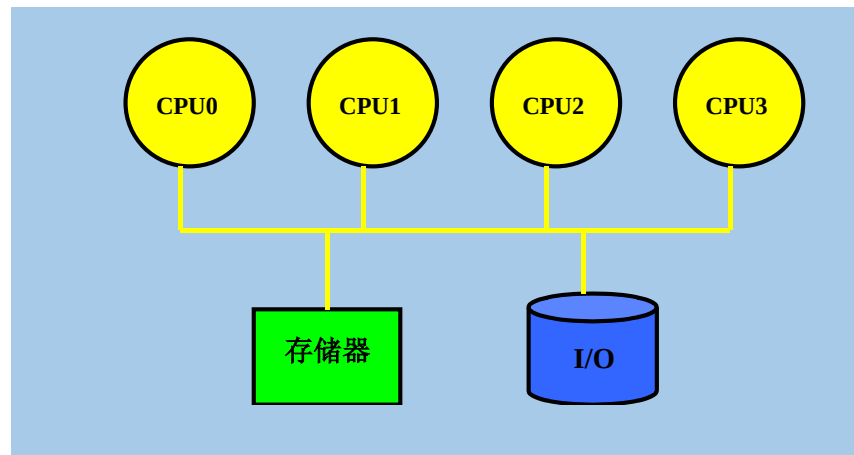
10.2 对称式共享存储器系统结构 (SMP/UMA)

- 多个处理器共享一个存储器。
- 当处理机规模较小时，这种计算机十分经济。
- 近些年，能在一个单独的芯片上实现2~8个处理器核。

例如：Sun公司 2006年 T1 8核的多处理器

- 支持对共享数据和私有数据的Cache缓存
私有数据供一个单独的处理器使用，
而共享数据则是供多个处理器使用。
- 共享数据进入Cache产生了一个新的问题

Cache的一致性问题



10.2 对称式共享存储器系统结构

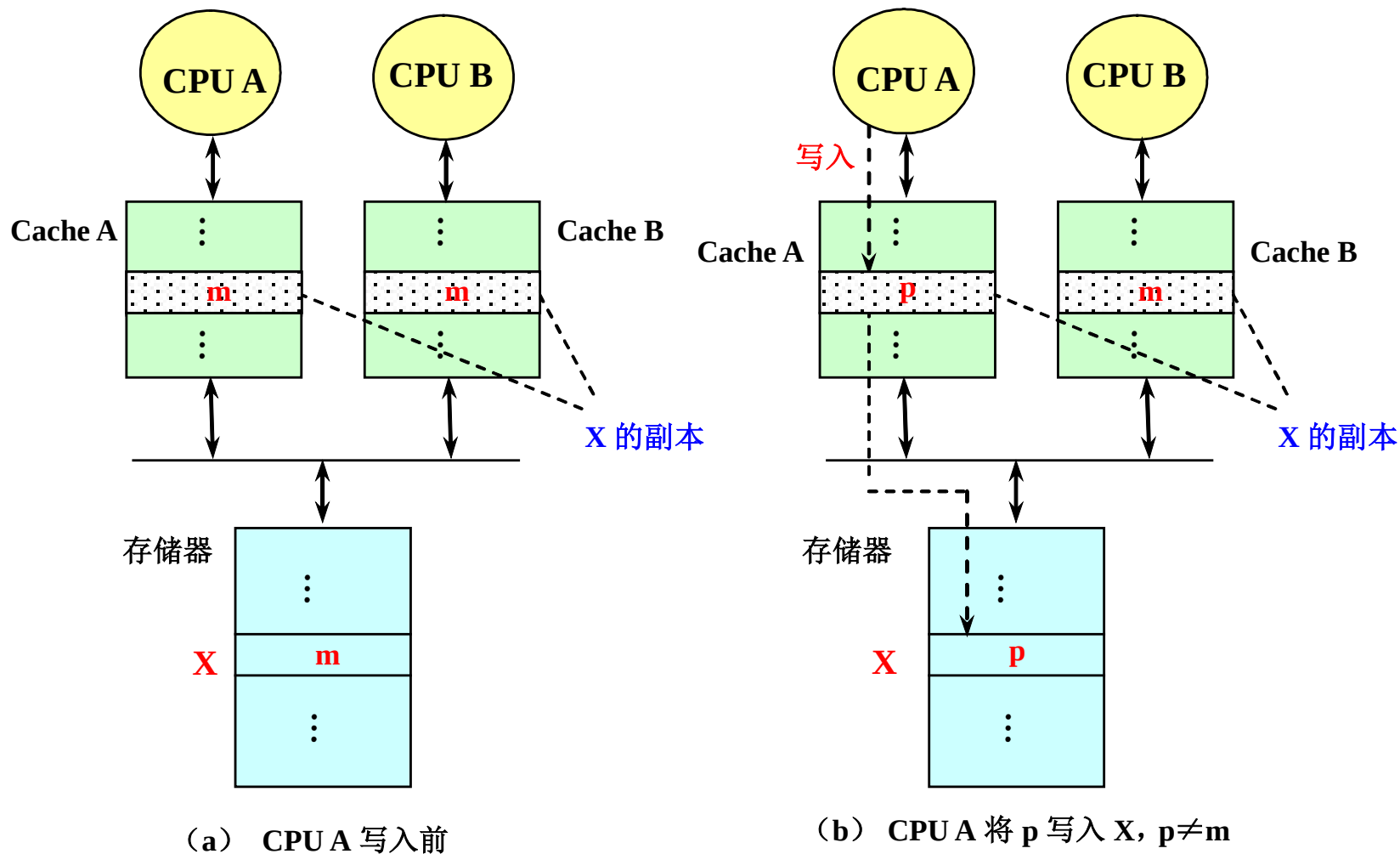
10.2.1 多处理机Cache一致性

1. 多处理机的Cache一致性（Cache Coherence）问题

- 允许共享数据进入Cache，就可能出现多个处理器的Cache中都有同一存储块的副本，
- 当其中某个处理器对其Cache中的数据进行修改后，就会使得其Cache中的数据与其他Cache中的数据不一致。

10.2 对称式共享存储器系统结构

例 由两个处理器（A和B）读写引起的Cache一致性问题



10.2 对称式共享存储器系统结构

2. 存储器的一致性

如果对某个数据项的任何读操作均可得到其最新写入的值，则认为这个存储系统是一致的。

➤ 存储系统行为的两个不同方面

- ❑ What: 读操作得到的是什么值
- ❑ When: 什么时候才能将已写入的值返回给读操作

10.2 对称式共享存储器系统结构

- 满足以下条件，则存储系统是一致的：
 - ❑ 处理器P对单元X进行写之后，又对单元X进行读，读和写之间没有其它处理器对单元X进行写，则P读到的值总是前面写进去的值。 —P写后P读
 - ❑ 处理器P对单元X进行写之后，另一处理器Q对单元X进行读，读和写之间无其它写，则Q读到的值应为P写进去的值。 —P写后Q读
 - ❑ 对同一单元的写是串行化的，即任意两个处理器对同一单元的两次写，从各个处理器的角度来看顺序都是相同的。（写串行化） —P写后Q写

10.2 对称式共享存储器系统结构

由于延迟，通常不能要求P对X写后就能在Q上读出。

- 在后面的讨论中，我们假设：
 - ❑ 直到所有的处理器均看到了写的结果，这个写操作才算完成；
 - ❑ 处理器的任何访存均不能改变写的顺序。就是说，允许处理器对读进行重排序，但必须以程序规定的顺序进行写。

10.2 对称式共享存储器系统结构

10.2.2 实现一致性的基本方案

在一致性的多处理机中，Cache提供两种功能：

- 共享数据的迁移（Migration）

把远程的共享数据复制一份迁入本地Cache。

减少了对共享数据的访问延迟和对共享存储器带宽的要求。

- 共享数据的复制（Replication）

把多个处理器要同时读取的共享数据在这些处理器本地Cache中各存放一个副本。

不仅减少了访问共享数据的延迟和对共享存储器带宽的要求，也减少了访问共享数据所产生的冲突。

一般情况下，小规模多处理机是采用硬件的方法来实现Cache的一致性。

10.2 对称式共享存储器系统结构

1. Cache一致性协议 (Cache Coherent Protocol)

在多个处理器中用来维护一致性的协议。

➤ **关键：**跟踪记录共享数据块的状态

➤ **两类协议：**

□ **目录式协议** (Directory)

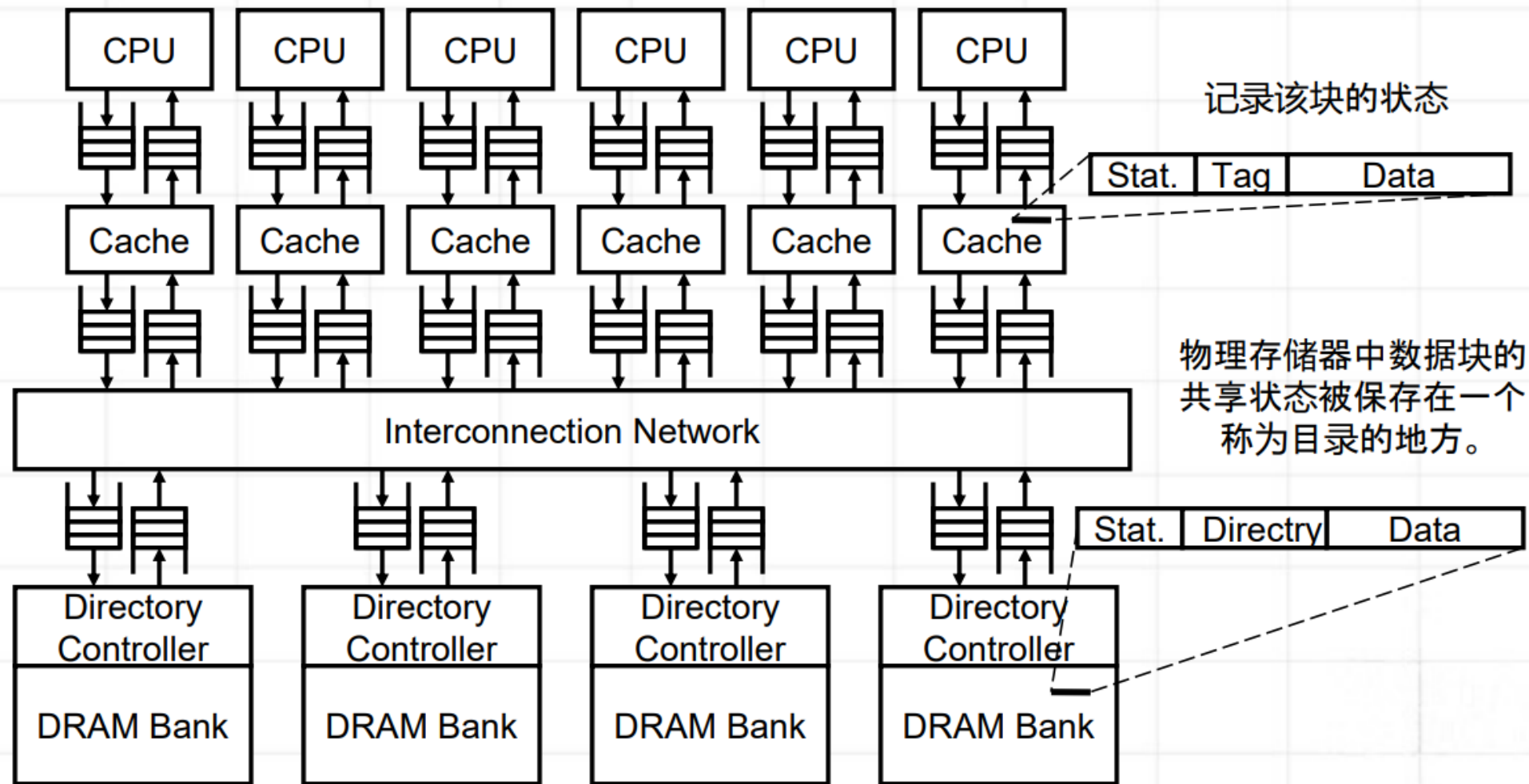
每个Cache的**共享状态**被保存在一个称为目录的地方。

□ **监听式协议** (Snooping)

每个Cache保留共享状态的副本，通过广播和监听来实现一致性。

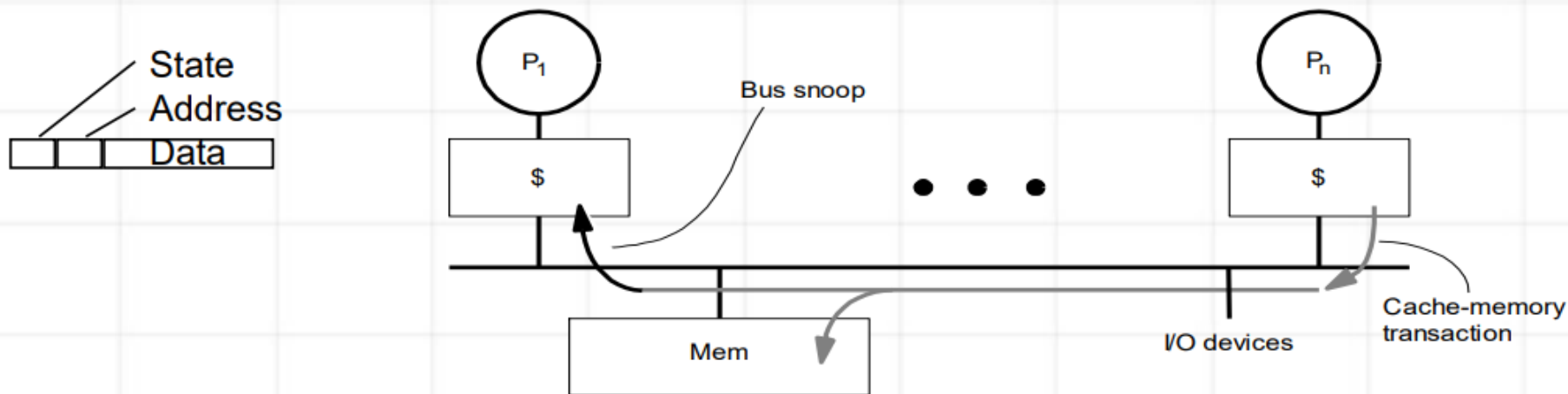
10.2 对称式共享存储器系统结构

目录式协议 (Directory)



10.2 对称式共享存储器系统结构

监听式协议（Snooping）



- Cache通常连在共享存储器的总线上，当某个Cache需要访问存储器时，它会请求放到总线上广播出去，其他各个Cache控制器通过监听总线（它们一直在监听）来判断它们是否有总线上请求的数据块。如果有，就进行相应的操作。
- 每个Cache除了包含物理存储器中块的数据拷贝之外，也保存着各个块的共享状态信息。

10.2 对称式共享存储器系统结构

2. 两种写更新操作方法来解决Cache一致性问题。

➤ 写作废协议

在处理器对某个数据项进行写入之前，保证它拥有对该数据项的唯一的访问权。(作废其它的副本)

应用广泛，对于监听或者目录方式皆可用。

➤ 写更新协议

当一个处理器对某数据项进行写入时，通过广播使其它Cache中所有对应于该数据项的副本进行更新。

10.2 对称式共享存储器系统结构

Cache更新后对存储器写策略:有写直达法和写回法两种

- 写直达Cache: 执行“写”操作时, 不仅写入Cache, 而且也写入下一级存储器。

因为所有写入的数据都同时被写回主存, 所以从主存中总可以取到其最新值。

- 写回Cache: 执行“写”操作时, 只写入Cache。仅当Cache中相应的块被替换时, 才写回主存。(设置“修改位”)

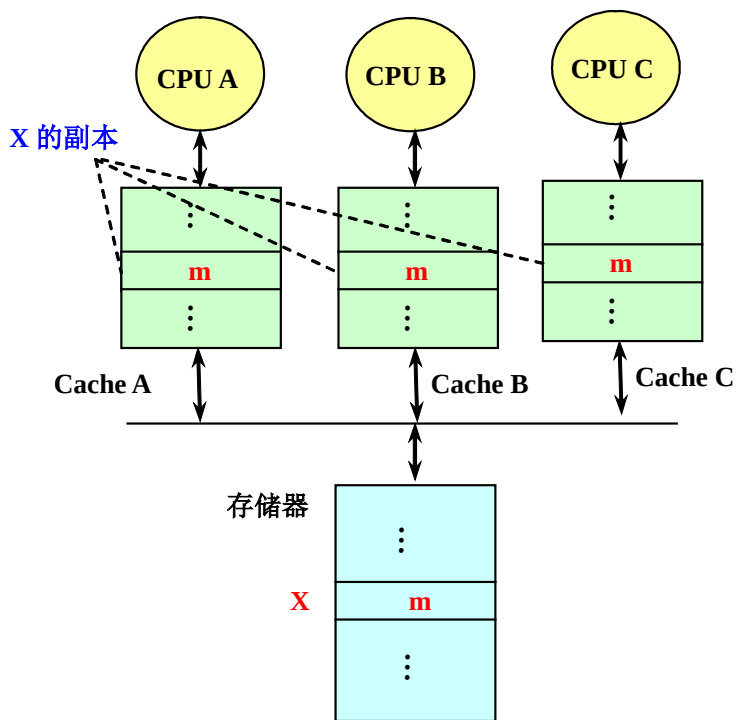
得到数据的最新值会困难一些, 因为最新值可能在某个Cache中, 也可能在主存中。

10.2 对称式共享存储器系统结构

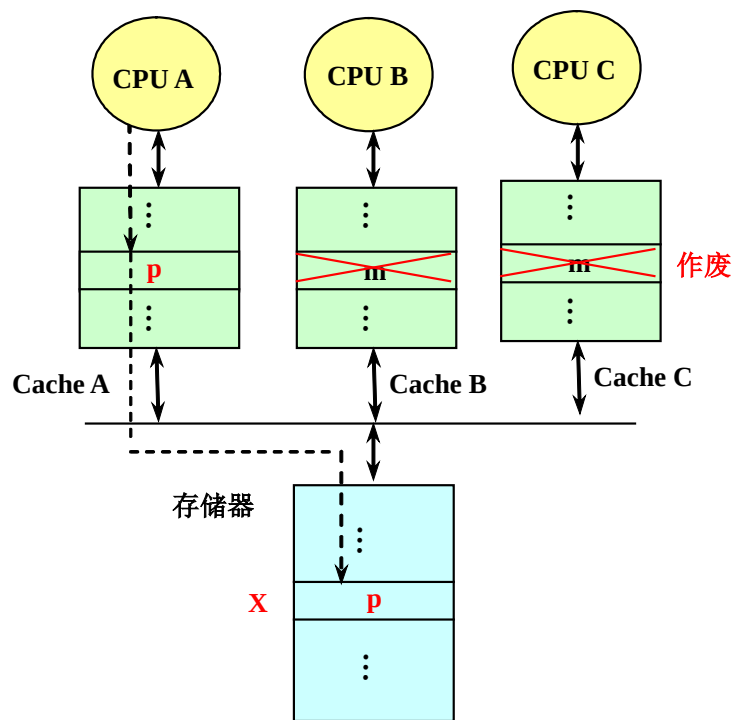
➤ 写作废协议

例 监听总线、写作废协议举例（采用写直达法）

初始状态： CPU A、CPU B、CPU C 都有 X 的副本。在 CPU A 要对 X 进行写入时，需先作废 CPU B 和 CPU C 中的副本，然后再将 p 写入 Cache A 中的副本，同时用该数据更新主存单元 X。



(a) CPU A 写入前



(b) CPU A 将 p 写入 X 后，作废其他 Cache 中的副本

10.2 对称式共享存储器系统结构

➤ 写更新协议

当一个处理器对某数据项进行写入时，通过广播使其它Cache中所有对应于该数据项的副本进行更新。

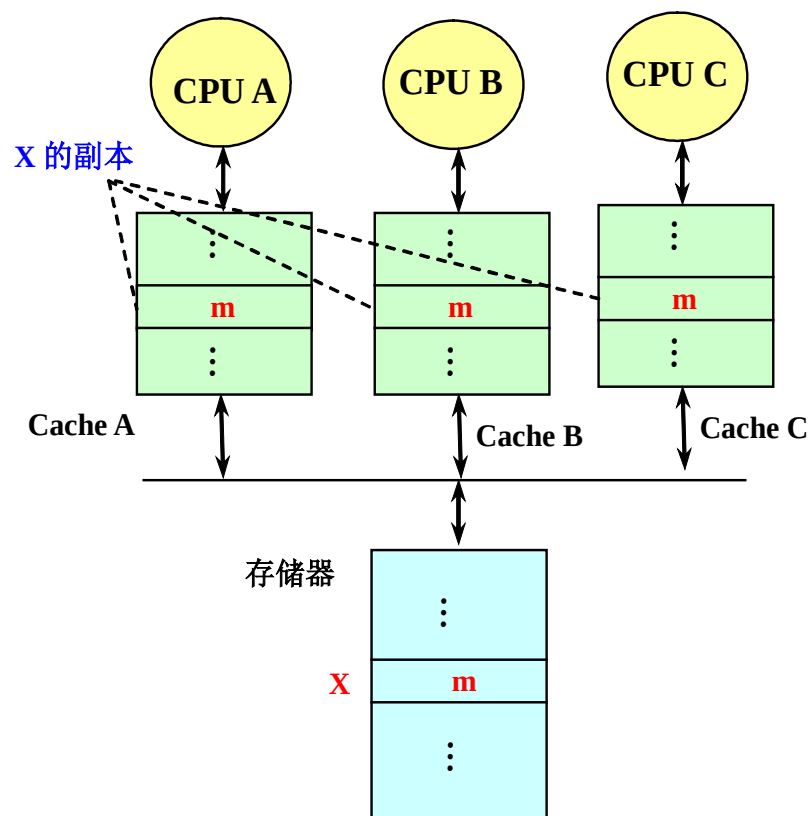
例 监听总线、写更新协议举例（采用写直达法）

假设：3个Cache都有X的副本。

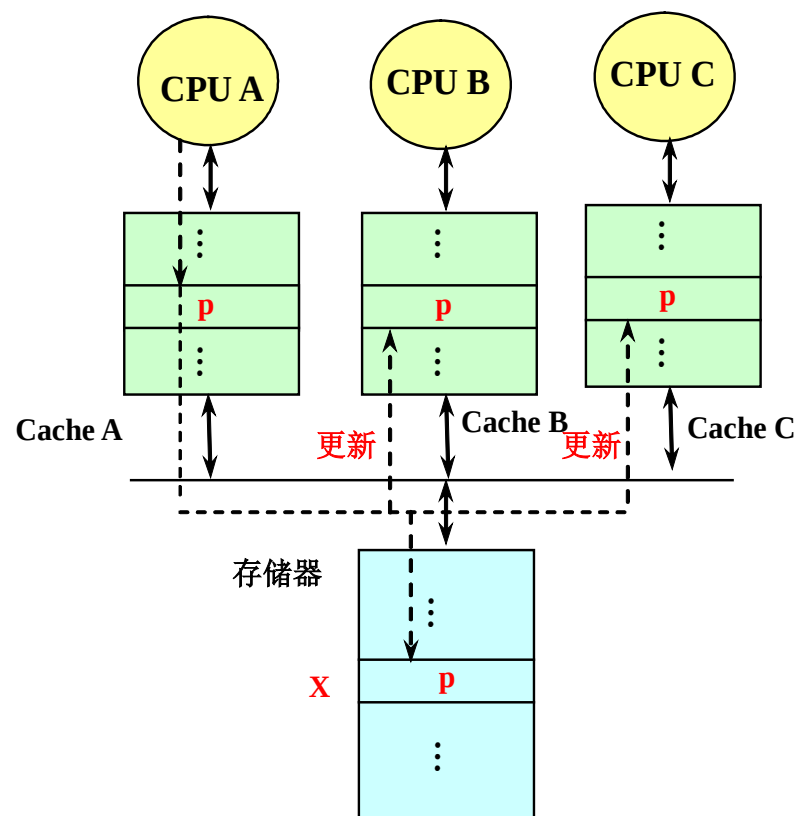
当CPU A将数据p写入Cache A中的副本时，将p广播给所有的Cache，这些Cache用p更新其中的副本。

由于这里是采用写直达法，所以CPU A还要将p写入存储器中的X。如果采用写回法，则不需要写入存储器。

10.2 对称式共享存储器系统结构



(a) CPU A 写入前



(b) CPU A 将 p 写入 X 后，更新其他 Cache 中的副本

10.2 对称式共享存储器系统结构

➤ 写更新和写作废协议性能上的差别主要来自：

- ❑ 在对同一个数据进行多次写操作而中间无读操作的情况下，写更新协议需进行多次写广播操作，而写作废协议只需一次作废操作。
- ❑ 在对同一Cache块的多个字进行写操作的情况下，写更新协议对于每一个写操作都要进行一次广播，而写作废协议仅在对该块的第一次写时进行作废操作即可。

写作废是针对Cache块进行操作，而写更新则是针对字（或字节）进行。

- ❑ 考虑从一个处理器A进行写操作后到另一个处理器B能读到该写入数据之间的延迟时间。

写更新协议的延迟时间较小，而写作废协议则需重新调入Cache。

在基于总线的多处理机中，总线和存储器带宽是紧缺资源，写作废协议耗费带宽少，故多采用写作废协议。

10.2 对称式共享存储器系统结构

10.2.3 监听协议的实现

1. 监听协议的基本实现技术

- 在每个结点内嵌入一个有限状态控制器。
 - ❑ 该控制器根据来自处理器或总线的请求以及Cache块的状态，做出相应的响应。
- 实现监听协议的关键有3个方面
 - ❑ 处理器之间通过一个可以实现广播的互连机制相连。
通常采用的是总线。
 - ❑ 当一个处理器的Cache响应本地CPU的访问时，如果它涉及到全局操作，其Cache控制器就要在获得总线的控制权后，在总线上发出相应的消息。
 - ❑ 所有处理器都一直在监听总线，它们检测总线上的地址在它们的Cache中是否有副本。若有，则响应该消息，并进行相应的操作
- 写操作的串行化：由总线实现 (获取总线控制权的顺序性)

10.2 对称式共享存储器系统结构

2. Cache发送到总线上的消息主要有以下两种：

- **RdMiss**——读不命中
 - **WtMiss**——写不命中
- 需要通过总线找到相应数据块的最新副本，然后调入本地Cache中。

10.2 对称式共享存储器系统结构

- 有的监听协议还增设了一条`Invalidate`消息，用来通知其他各处理器作废其Cache中相应的副本。
 - 与`WtMiss`的区别：`Invalidate`不引起调块
 - Cache的标识（`tag`）可直接用来实现监听。
 - 作废一个块只需将其`有效位置`为无效。
 - 给每个Cache块增设一个`共享位`
 - 为“1”：该块是被多个处理器所共享
 - 为“0”：仅被某个处理器所独占
- 块的拥有者：**拥有该数据块的唯一副本的处理器。

10.2 对称式共享存储器系统结构

3. 监听协议举例

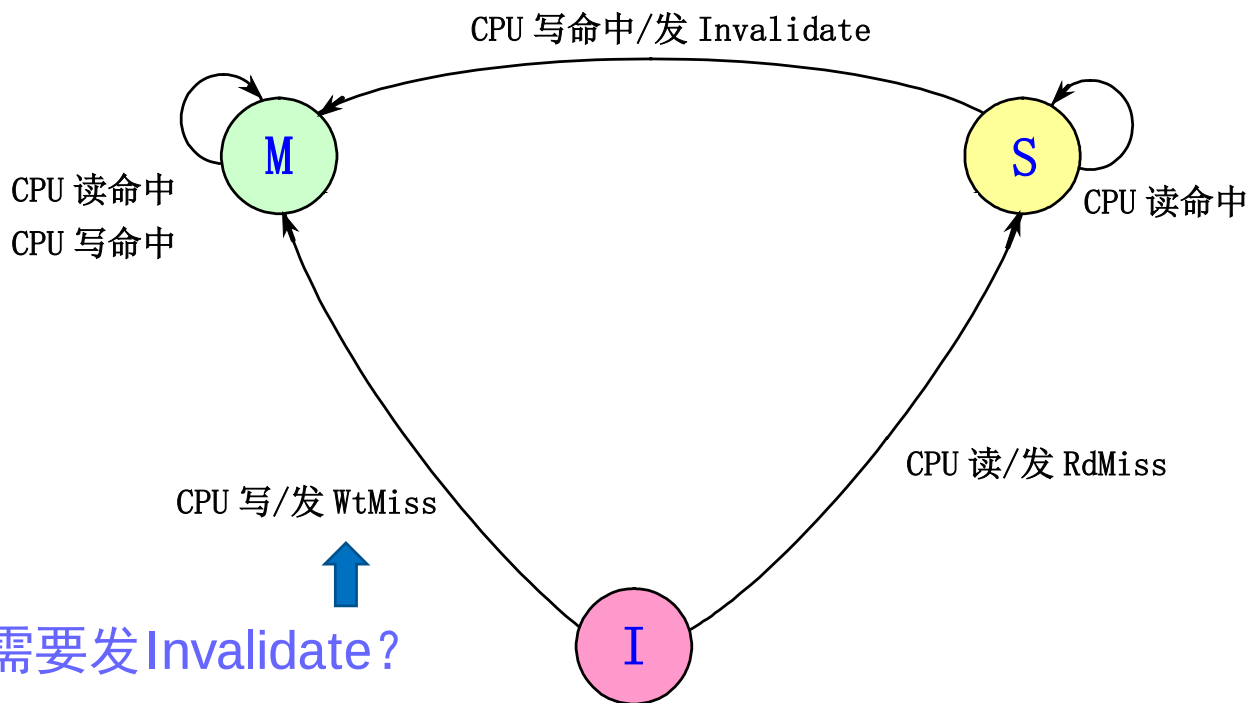
- 在每个结点内嵌入一个有限状态控制器。
 - 该控制器根据来自处理器或总线的请求以及Cache块的状态，做出相应的响应。
- 每个数据块的状态取以下3种状态中的一种：
 - 无效（简称I，Invalid）：Cache中该块的内容为无效。
 - 共享（简称S，Shared）：该块可能处于共享状态。
 - 在多个处理器中都有副本。这些副本都相同，且与存储器中相应的块相同。
 - 已修改（简称M，Modified）：该块已经被修改过，并且还没写入存储器。
 - （块中的内容是最新的，系统中唯一的最新副本）

10.2 对称式共享存储器系统结构

下面来讨论在各种情况下监听协议所进行的操作。

➤ 响应来自处理器的请求

□ 不发生替换的情况

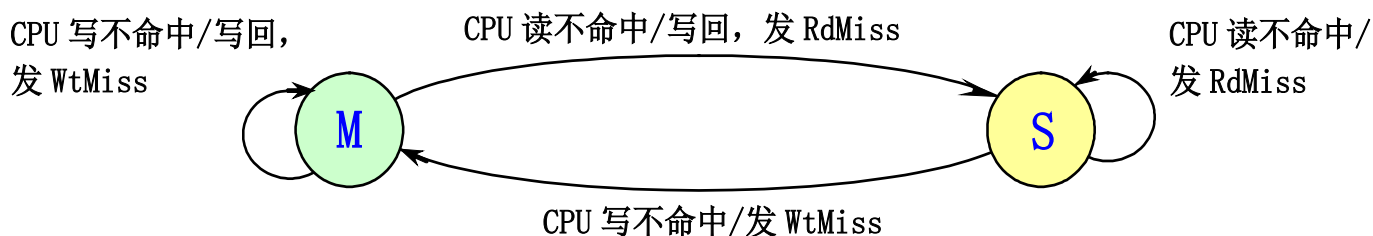


写作废协议中（采用写回法），Cache块的状态转换图

10.2 对称式共享存储器系统结构

□ 发生替换的情况

当前数据块让出空间要被其他数据块替换。

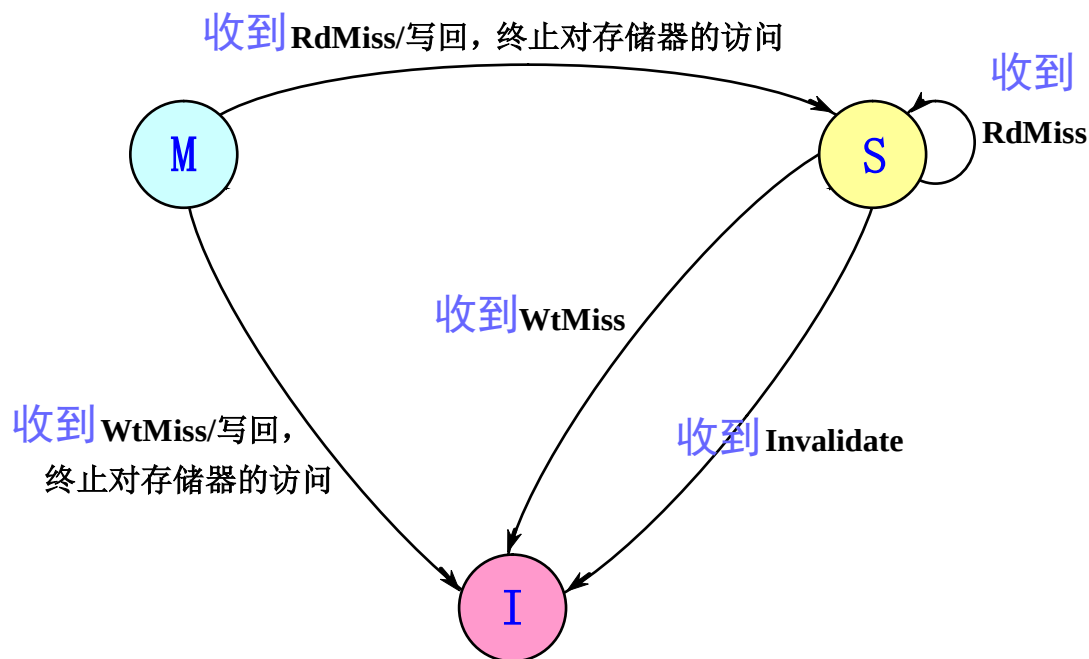


写作废协议中（采用写回法），Cache块的状态转换图

操作具有原子性（Atomic）：操作进行过程中不能被打断。

➤ 响应来自总线的请求

- 每个处理器都在监视总线上的消息和地址，当发现有与总线上的地址相匹配的**Cache**块时，就要根据该块的状态以及总线上的消息，进行相应的处理。



写作废协议中（采用写回法），Cache块的状态转换图

10.3 分布式共享存储器系统结构(DSM/NUMA)

10.3.1 目录协议的基本思想

- 广播和监听的机制使得监听一致性协议的可扩展性很差。
- 寻找替代监听协议的一致性协议。



（采用目录协议）

1. 目录协议

- **目录：**一种集中的数据结构。对于存储器中的每一个可以调入Cache的数据块，在目录中设置一条目录项，用于记录该块的状态以及哪些Cache中有副本等相关信息。

10.3 分布式共享存储器系统结构

- **特点:**

对于任何一个数据块，都可以快速地在唯一的一个位置中找到相关的信息。这使一致性协议避免了广播操作。

- **位向量:** 记录哪些Cache中有副本。

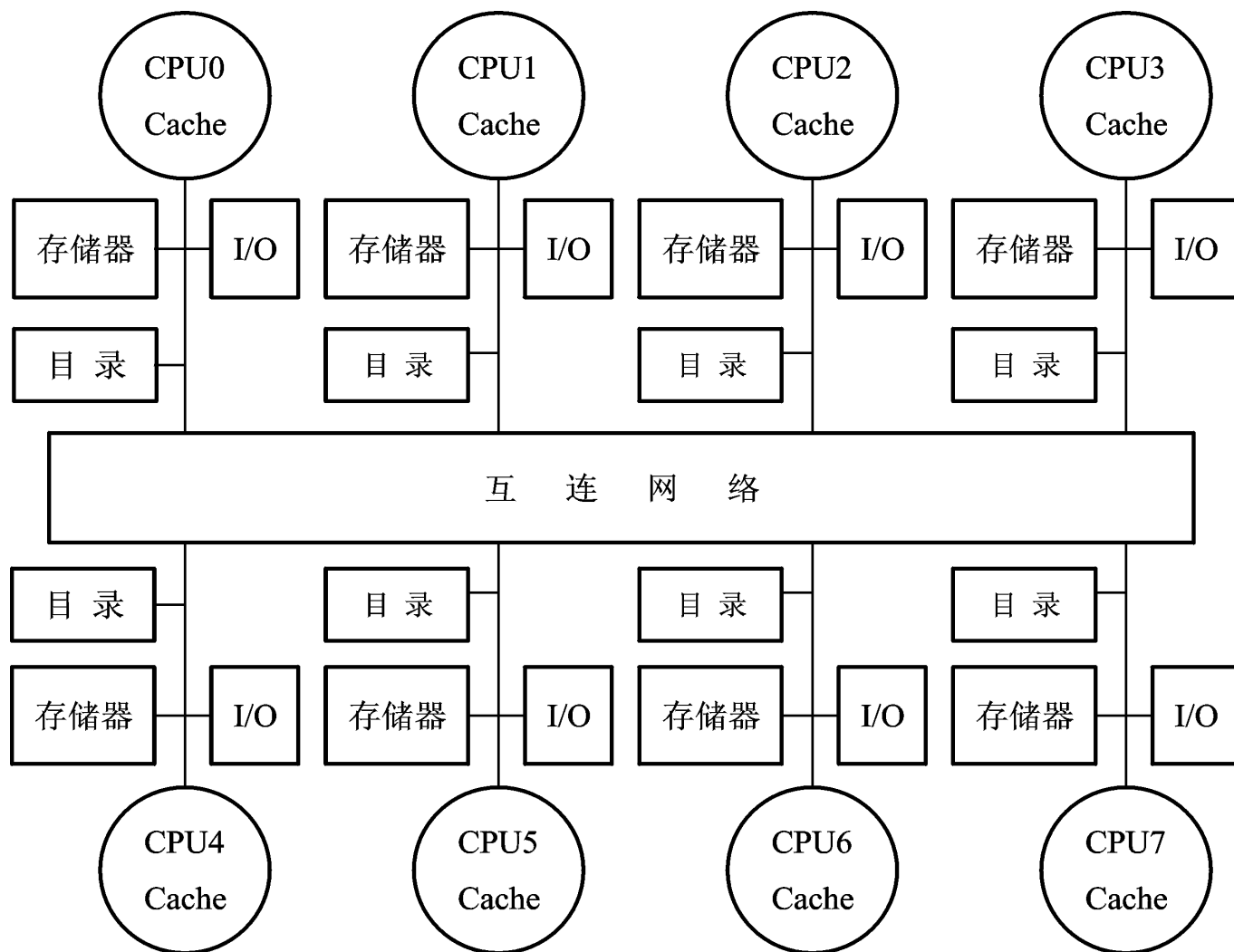
- 每一位对应于一个处理器。“1/0”表示相应处理器的Cache有/无副本。
- 长度与处理器的个数成正比。
- 由位向量指定的处理机的集合称为**共享集S**。

- **分布式目录**

- 目录与存储器一起分布到各结点中，从而对于不同目录内容的访问可以在不同的结点**并行**进行。
- 每个结点的目录中的信息是对应于该结点存储器中的数据块。

10.3 分布式共享存储器系统结构

对每个结点增加目录后的分布式存储器多处理机



10.3 分布式共享存储器系统结构

- 目录法最简单的实现方案：对于存储器中每一块都在目录中设置一项。目录中的信息量与 $M \times N$ 成正比。

其中：

- M ：存储器中存储块的总数量
- N ：处理器的个数
- 由于 $M = K \times N$ ， K 是每个处理机中存储块的数量，所以如果 K 保持不变，则目录中的信息量就与 N^2 成正比。



扩充性并不好

10.3 分布式共享存储器系统结构

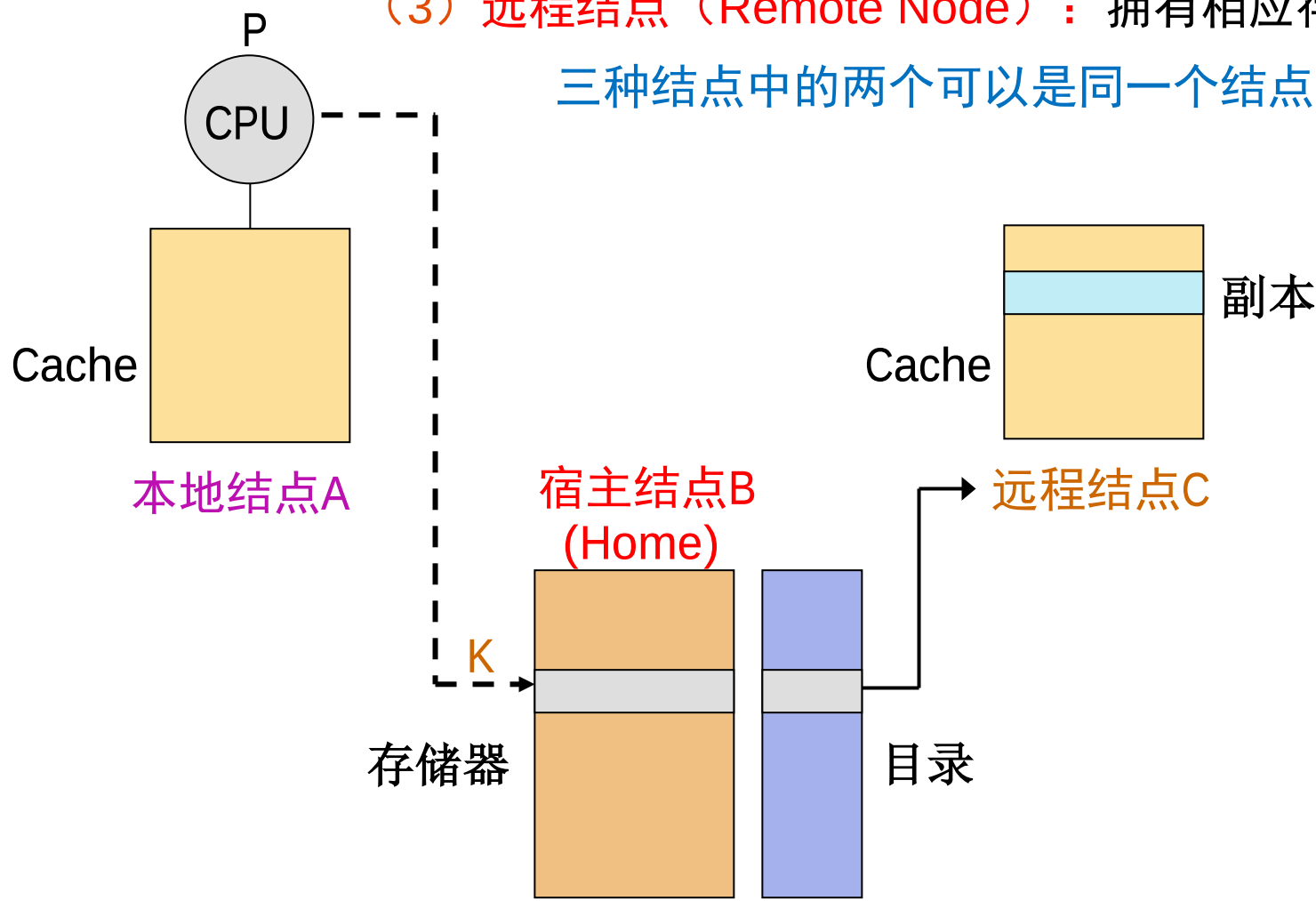
2. 在目录协议中，存储块的状态有3种：

- **未缓冲（Uncached）**：该块尚未被调入Cache。所有处理器的Cache中都没有这个块的副本。
- **共享（Shared）**：该块在一个或多个处理机上有这个块的副本，且这些副本与存储器中的该块相同。
- **独占（Exclusive）**：仅有一个处理机有这个块的副本，且该处理机已经对其进行了写操作，所以其内容是最新的，而存储器中该块的数据已过时。
这个处理机称为该**块的拥有者**。

10.3 分布式共享存储器系统结构

3. 三种结点：(1) 本地结点 (Local Node)：发出访问请求的结点
(2) 宿主结点 (Home Node)：访问的存储单元的结点
(3) 远程结点 (Remote Node)：拥有相应存储块副本的结点。

三种结点中的两个可以是同一个结点，协议不变。



10.3 分布式共享存储器系统结构

4. 在结点之间发送的消息

➤ 本地结点发给宿主结点（目录）的消息

说明：括号中的内容表示所带参数。

P：发出请求的处理机编号，**K**：所要访问的地址

□ **RdMiss (P, K)**

处理机**P**读取地址为**K**的数据时不命中，请求宿主结点提供数据（块），并要求把**P**加入共享集。

□ **WtMiss (P, K)**

处理机**P**对地址**K**进行写入时不命中，请求宿主结点提供数据，并使**P**成为所访问数据块的独占者。

□ **Invalidate (K)**

请求向所有拥有相应数据块副本（包含地址**K**）的远程**Cache**发**Invalidate**消息，作废这些副本。

10.3 分布式共享存储器系统结构

➤ 宿主结点（目录）发送给远程结点的消息

❑ Invalidate (K)

作废远程Cache中包含地址K的数据块。

❑ Fetch (K)

从远程Cache中取出包含地址K的数据块，并将之送到宿主结点。把远程Cache中那个块的状态改为“共享”。

❑ Fetch&Inv (K)

从远程Cache中取出包含地址K的数据块，并将之送到宿主结点。然后作废远程Cache中的那个块。

10.3 分布式共享存储器系统结构

➤ 宿主结点发送给本地结点的消息

$DReply(D)$

- D 表示数据内容。
- 把从宿主存储器获得的数据返回给本地Cache。

➤ 远程结点发送给宿主结点的消息

$WtBack(K, D)$

- 把远程Cache中包含地址 K 的数据块写回到宿主结点中，该消息是远程结点对宿主结点发来的“取数据”或“取/作废”消息的响应。

10.3 分布式共享存储器系统结构

➤ 本地结点发送给被替换块的宿主结点的消息

--- Cache新导入一个块，需要替换一个既存的块

□ MdSharer (P, K)

用于当本地Cache中需要替换一个包含地址K的块、且该块未被修改过的情况。这个消息发给该块的宿主结点，请求它将P从共享集中删除。如果删除后共享集变为空集，则宿主结点还要将该块的状态改变为“未缓存”（U）。

□ WtBack2 (P, K, D)

用于当本地Cache中需要替换一个包含地址K的块、且该块已被修改过的情况。这个消息发给该块的宿主结点，完成两步操作：①把该块写回；②进行与MdSharer相同的操作。

10.3 分布式共享存储器系统结构

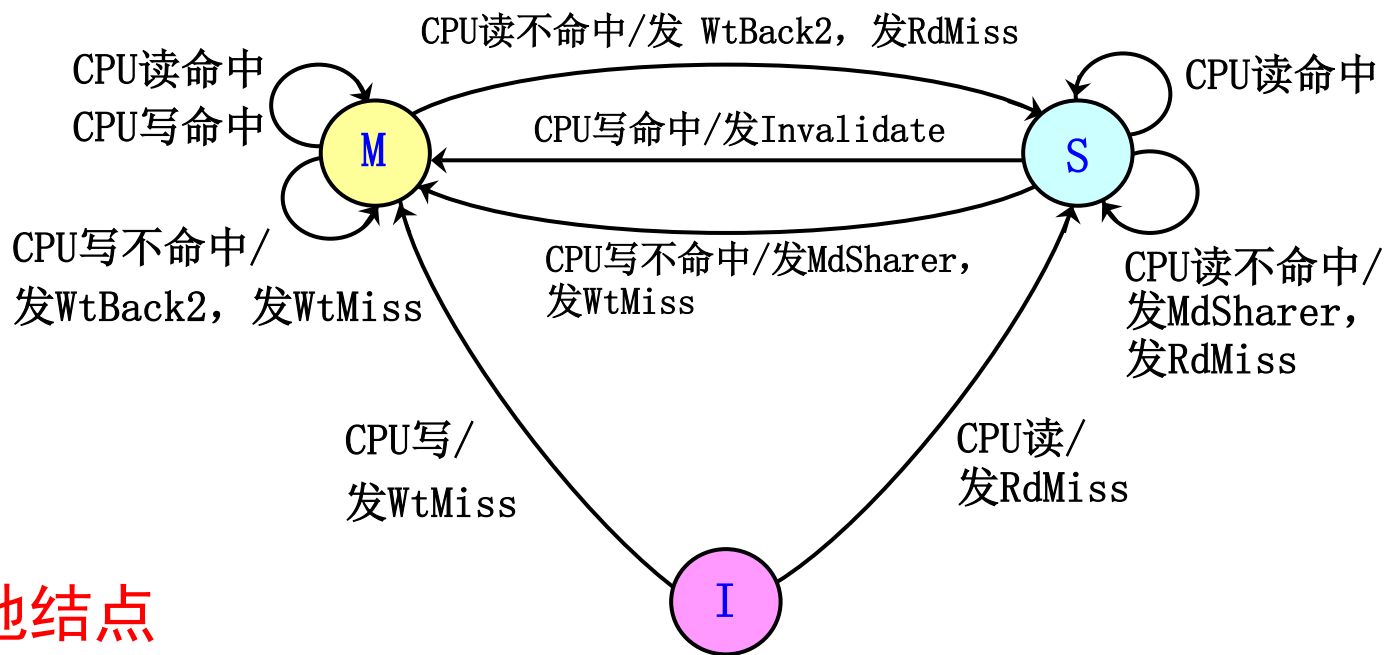
10.3.2 目录协议实例

- 在基于目录的协议中，目录承担了一致性协议操作的主要功能。
 - 本地结点把请求发给宿主结点中的目录，再由目录控制器有选择地向远程结点发出相应的消息。
 - 发出的消息会产生两种不同类型的动作：
 - 更新目录状态
 - 使远程结点完成相应的操作

10.3 分布式共享存储器系统结构

1. 在基于目录协议的系统中，Cache块的状态转换图。

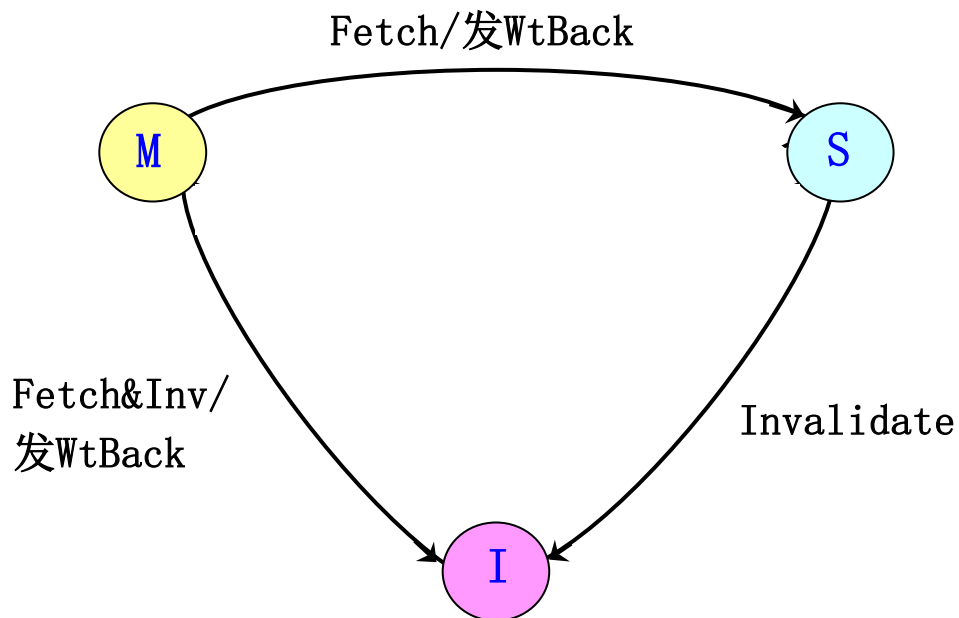
➤ 响应本地Cache CPU请求



Cache块的状态转换图（响应本地CPU请求）

10.3 分布式共享存储器系统结构

- 远程结点中Cache块响应来自宿主结点的请求的状态转换图



远程结点

Cache块的状态转换图（响应宿主目录请求）

10.3 分布式共享存储器系统结构

2. 目录的状态转换及相应的操作

- 目录中存储器块的三种状态：
 - ❑ 未缓存（**UnCached**）
 - ❑ 共享（**Shared**）
 - ❑ 独占（**Exclusive**）
- **位向量**记录拥有其副本的处理器集合——**共享集合**。
- 对于从本地结点发来的请求，目录所进行的操作包括：
 - ❑ 向远程结点发送消息以完成相应的操作。这些远程结点由共享集合指出；
 - ❑ 修改目录中该块的状态；
 - ❑ 更新共享集合。

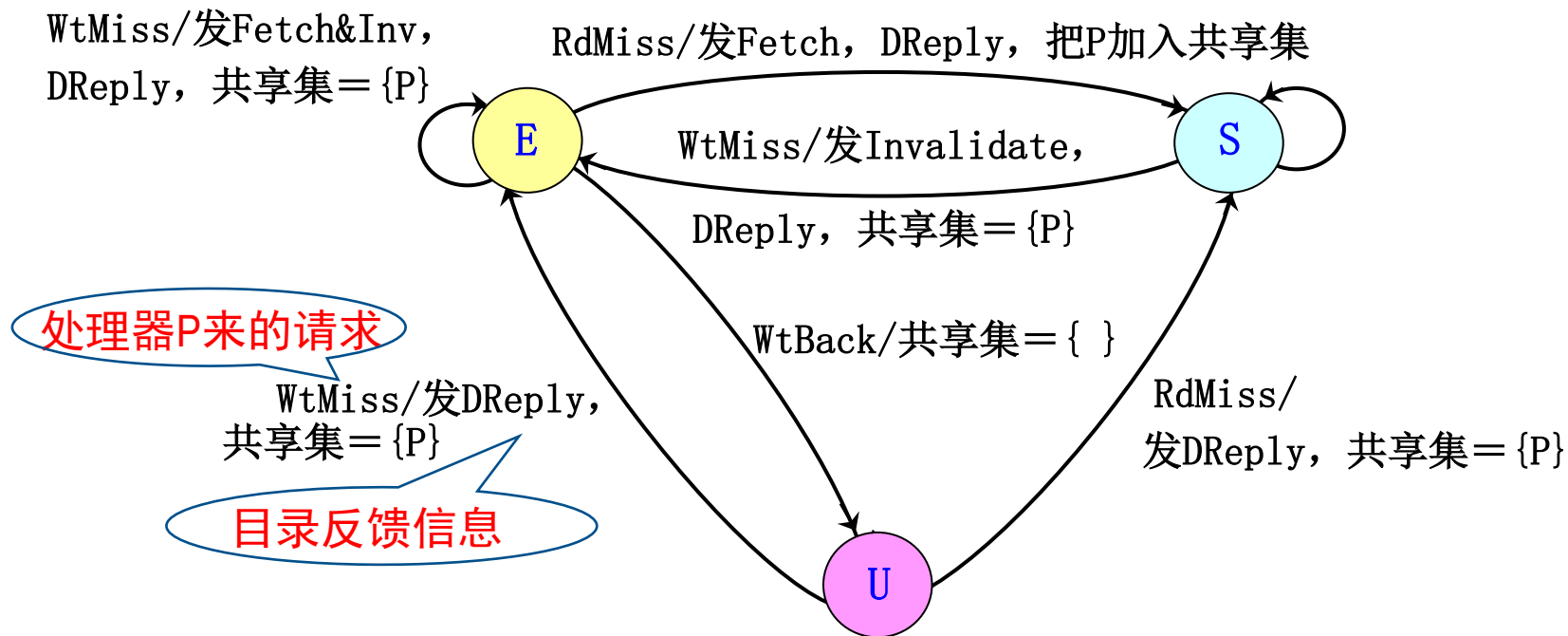
10.3 分布式共享存储器系统结构

➤ 目录可能接收到3种不同的请求

- ❑ 读不命中（RdMiss）
- ❑ 写不命中（WtMiss）
- ❑ 数据写回（WtBack2）

（假设这些操作是原子的）

10.3 分布式共享存储器系统结构



U--未缓存 (Uncached)
E--独占 (Exclusive) : 可读写

S--共享 (Shared) : 只读
P--发起请求处理器

宿主结点

目录的状态转换及相应的操作

10.3 分布式共享存储器系统结构

- 当一个块处于未缓存状态时，对该块发出的请求及处理操作为：
 - RdMiss（读不命中）
 - 将所要访问的存储器数据送往请求方处理机，且该处理机成为该块的唯一共享结点，本块的状态变成共享。
 - WtMiss（写不命中）
 - 将所要访问的存储器数据送往请求方处理机，该块的状态变成独占，表示该块仅存在唯一的副本。其共享集合仅包含该处理机，指出该处理机是其拥有者。

10.3 分布式共享存储器系统结构

- 当一个块处于**共享状态**时，其在存储器中的数据是当前最新的，对该块发出的请求及其处理操作为：
 - **RdMiss**（读不命中）
 - 将存储器数据送往请求方处理机，并将其加入共享集合。
 - **WtMiss**（写不命中）
 - 将数据送往请求方处理机，对共享集合中所有的处理机发送作废消息，且将共享集合改为仅含有该处理机，该块的状态变为独占。

10.3 分布式共享存储器系统结构

- 当某块处于**独占状态**时，该块的最新值保存在共享集合所指出的唯一处理机（拥有者）中。

有三种可能的请求：

- **RdMiss**（读不命中）

- 将“取数据”的消息发往拥有者处理机，将它所返回给宿主结点的数据写入存储器，并进而把该数据送回请求方处理机，将请求方处理机加入共享集合。
- 此时共享集合中仍保留原拥有者处理机（因为它仍有一个可读的副本）。
- 将该块的状态变为共享。

10.3 分布式共享存储器系统结构

□ WtMiss（写不命中）

- 该块将有一个新的拥有者。
- 给旧的拥有者处理机发送消息，要求它将数据块送回宿主结点写入存储器，然后再从该结点送给请求方处理机。
- 同时还要把旧拥有者处理机中的该块作废。把请求处理机加入共享者集合，使之成为新的拥有者。
- 该块的状态仍旧是独占。

□ WtBack（写回）

- 当一个块的拥有者处理机要从其Cache中把该块替换出去时，必须将该块写回其宿主结点的存储器中，从而使存储器中相应的块中存放的数据是最新的（宿主结点实际上成为拥有者）；
- 该块的状态变成未缓冲，其共享集合为空。

10.3 分布式共享存储器系统结构

10.3.3 目录的三种结构

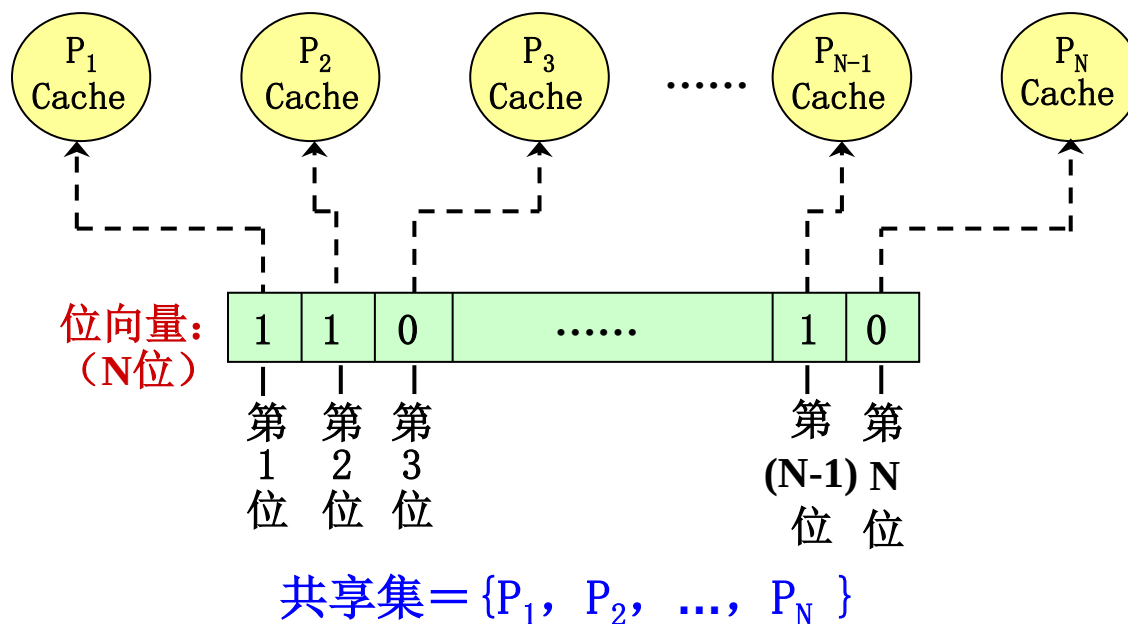
- 不同目录协议的**主要区别**主要有两个
 - 所设置的存储器块的状态及其个数不同
 - 目录的结构
- 目录协议分为**3**类
 - 全映象（Full-mapped）目录、
 - 有限映象（Limited-mapped）目录、
 - 链式（Chained）目录

10.3 分布式共享存储器系统结构

1. 全映象目录

- 每一个目录项都包含一个 N 位（ N 为处理机的个数）的位向量，其每一位对应于一个处理机。
 - 举例
- 优点：处理比较简单，速度也比较快。
- 缺点：
 - 存储空间的开销很大。
 - 目录项的数目与处理机的个数 N 成正比，而目录项的大小（位数）也与 N 成正比，因此目录所占用的空间与 N^2 成正比。
 - 可扩放性很差。

10.3 分布式共享存储器系统结构



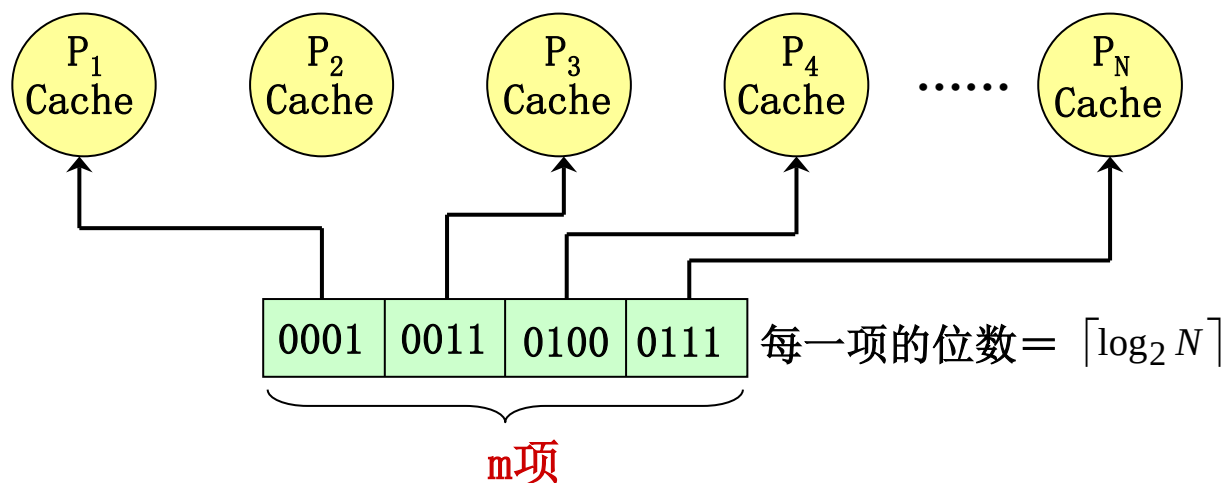
- 当位向量中的值为“1”时，就表示它所对应的处理机有该数据块的副本；否则就表示没有。
- 在这种情况下，共享集合由位向量中值为“1”的位所对应的处理机构成。

10.3 分布式共享存储器系统结构

2. 有限映像目录

- 提高其可扩充性和减少目录所占用的空间。
- **核心思想：**采用位数固定的目录项目
 - 限制同一数据块在所有Cache中的副本总数。
 - 例如，限定为常数 m 。则目录项中用于表示共享集合所需的二进制位数为： $m \times \lceil \log_2 N \rceil$ 。
 - 目录所占用的空间与 $N \times \lceil \log_2 N \rceil$ 成正比。
- **缺点**
 - 当同一数据副本个数大于 m 时，需要驱逐一个。

10.3 分布式共享存储器系统结构



共享集 = $\{P_1, P_3, P_4, P_7\}$

有限映像目录 ($m=4$, $N \geq 8$ 的情况)

10.3 分布式共享存储器系统结构

3. 链式目录

- 用一个目录指针链表来表示共享集合。当一个数据块的副本数增加（或减少）时，其指针链表就跟着变长（或变短）。
- 由于链表的长度不受限制，因而带来了以下优点：既不限制副本的个数，又保持了可扩展性。

10.3 分布式共享存储器系统结构

➤ 链式目录有两种实现方法

▣ 单链法

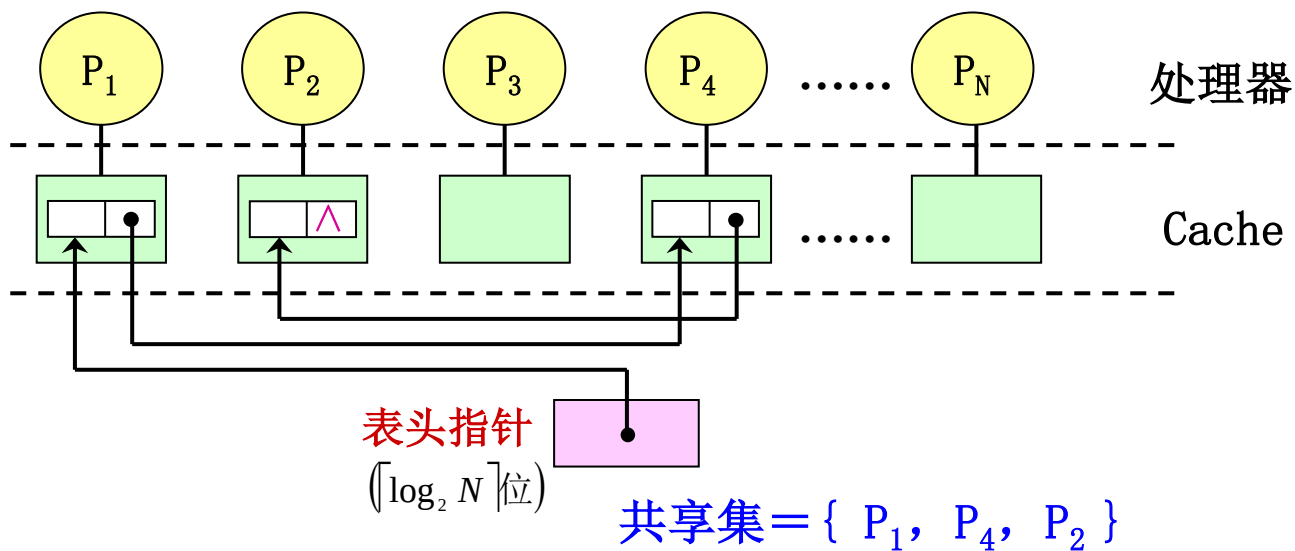
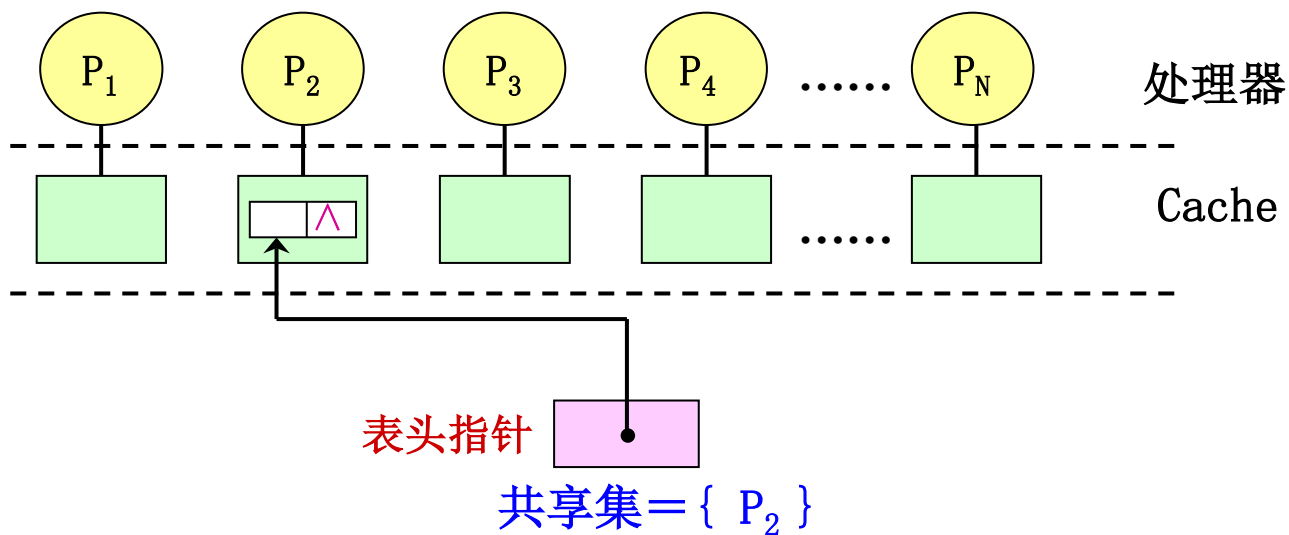
当Cache中的块被替换出去时，需要对相应的链表进行操作——把相应的链表元素（假设是链表中的第*i*个）删除。实现方法有以下两种：

- 沿着链表往下寻找第*i*个元素，找到后，修改其前后的链接指针，跳过该元素。
- 找到第*i*个元素后，作废它及其后的所有元素所对应的Cache副本。

▣ 双链法

- 在替换时不需要遍历整个链表。
- 节省了处理时间，但其指针增加了一倍，而且一致性协议也更复杂了。

采用单向链法的示意图



小结 多处理机 (Multi processor) -- MIMD

1. 存储结构

集中式：对称式共享存储器系统结构 (SMP)

分布式：{ 共享存储器系统结构 (DSM)

私有存储器

} 统一地址空间

独立地址空间

2. Cache一致性

监听协议，目录协议

写更新处理：写作废协议、写更新协议

写策略：写直达、写回

3. 通信机制

➤ 共享存储器通信机制 (load和store)

➤ 消息传递通信机制

4. 内存访问的UMA架构 (SMP) 与NUMA架构 (DSM)